



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

16-课程回顾

贾弘洋 & 刘勇攀

2025/6/3



/ 授课计划 (Tentative)



- 1) Introduction of DNN
- 2) DNN的原理
- 3) Quantization (Lab 1 out)
- 4) Pruning (剪枝)
- 5) Kernel Computation & Computing Flow
- 6) Lab 1 Presentation
- 7) Accelerator & Case Study (TPU&Ascend) (Lab 2 out)
- 8) Mapping of DNN & Toolchain (Layer Fusion)
- 9) Data Movement & Storage Hierarchy (Compression)
- 10) 校庆+五一
- 11) Lab 2 Presentation
- 12) Codesign Sparse-aware Arch & NAS (Lab 3 out)
- 13) Compute-in Memory (CIM)
- 14) End2End Deployment & ALP
- 15) Training
- 16) Conclusion

深度学习神经网络的训练和推理

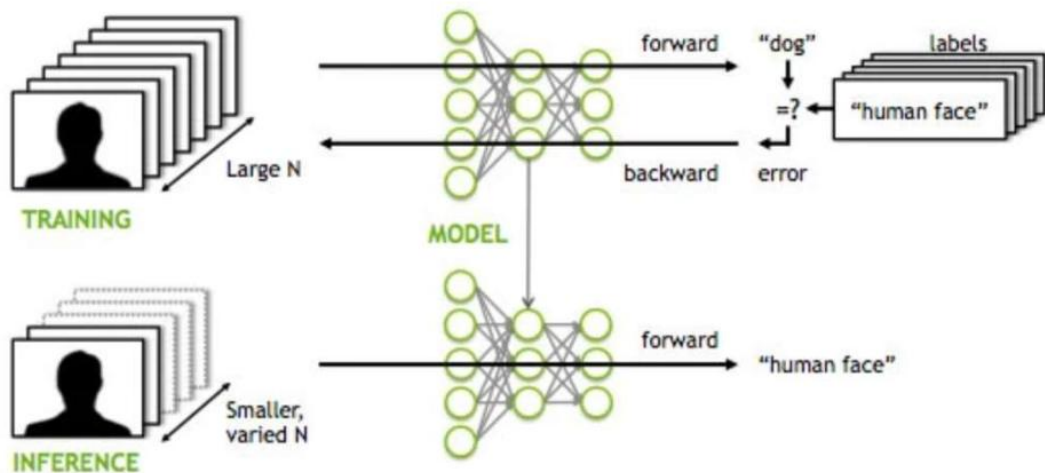


- Training

- Dataset
- Cost function
- Optimization function
- Model

- Inference

- Dataset
- Model



- **Model**
- Dataset
 - Problem dependent
 - Training set, validation set, test set
- Cost function
 - Convex function
 - Backpropagation (using chain rule)
- Optimization

/ Network结构的发展与讨论



- 分类任务
 - AlexNet
 - VGG-net
 - ResNet
- 检测任务
 - Two-stage model
 - One-stage model
- 序列模型
 - RNN
 - LSTM
 - Transformers



- Fixed-point and floating-point representation
- Hardware implications
- Quantization for deep learning
- Classic research for quantization methods
- State-of-the-art hardware support for low precision DNNs

/ Quantization for deep learning

- Two mainstream approaches
 - **Post-training quantization**: quantize a pre-trained model to maintain its accuracy. Actually, it is very challenging because FP32 has far larger representation range and higher precision than 8-bit fixed-point.
 - **Quantization-aware training**: train to get a quantized model. It is even more challenging.

/Post-training Quantization

- Quantization scheme:

- The correspondence between the fixed-point representation of values, i.e., **“q” for “quantized value”** and their floating-point value, i.e., **“r” for “real value”**.
- Recall “slope and bias” of fixed-point representation: $y = s \cdot x + z$

$$r = S(q - Z)$$

```
template<typename QType> // e.g. QType=uint8
struct QuantizedBuffer {
    vector<QType> q;      // the quantized values
    float S;             // the scale
    QType Z;             // the zero-point
};
```

r: real floating-point value
q: quantized fixed-point value
S: scaling factor
Z: zero point (bias)

/ Post-training Quantization

- Quantization scheme

$$OA[i, k] = \sum_{j=1}^N (W[i, j] * IA[j, k])$$

↑ ↑ ↙ input
output Weight in DNN

$$r = S(q - Z)$$

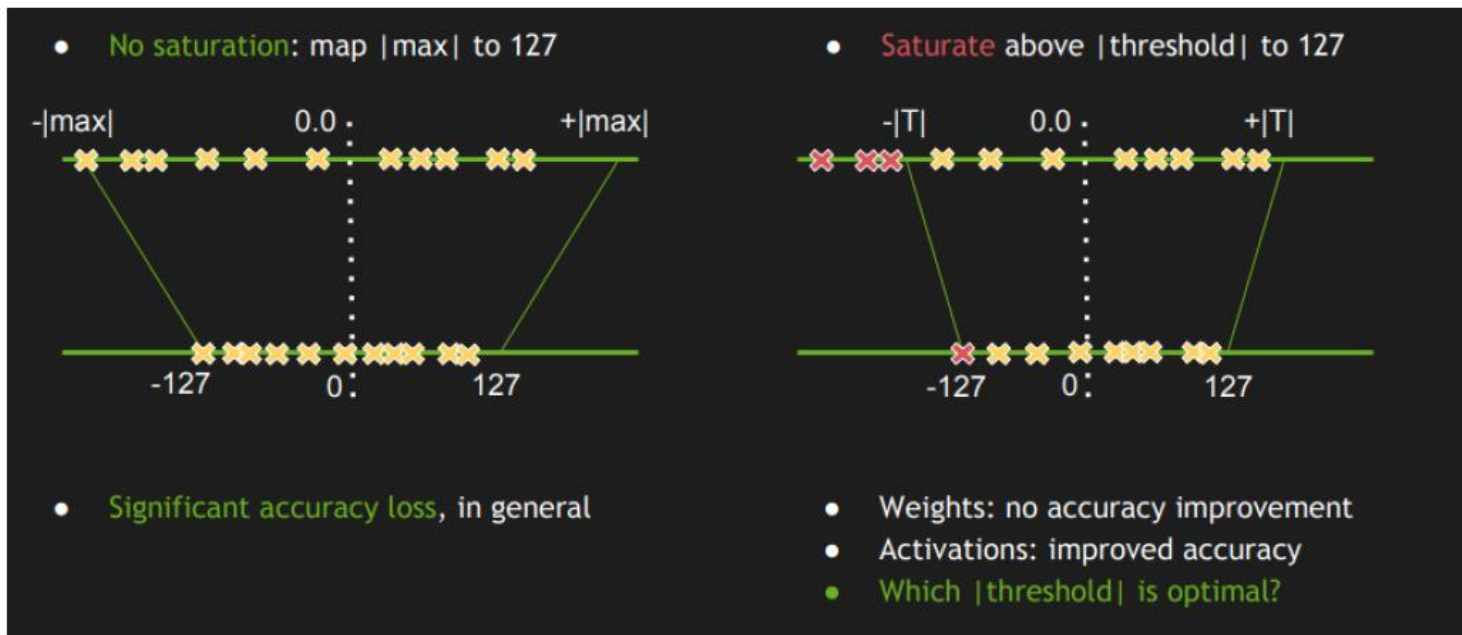
$$S_{OA} \left(q_{OA}^{(i,k)} - Z_{OA} \right) = \sum_{j=1}^N \left(S_W \left(q_W^{(i,j)} - Z_W \right) * \left(S_{IA} \left(q_{IA}^{(j,k)} - Z_{IA} \right) \right) \right)$$

$$q_{OA}^{(i,k)} = Z_{OA} + \frac{S_W * S_{IA}}{S_{OA}} \sum_{j=1}^N \left(\left(q_W^{(i,j)} - Z_W \right) * \left(q_{IA}^{(j,k)} - Z_{IA} \right) \right)$$

$$q_{OA}^{(i,k)} = \frac{S_W * S_{IA}}{S_{OA}} \sum_{j=1}^N \left(q_W^{(i,j)} * q_{IA}^{(j,k)} \right) \quad \text{(If we neglect the zero point (bias))}$$

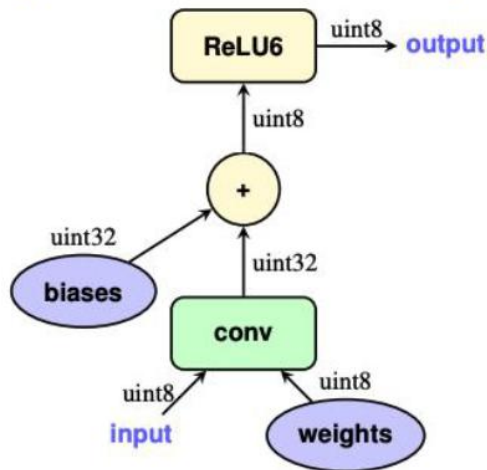
/ Post-training Quantization

- The essential problem: how can we determine the scaling factor of input, output and weight in quantizing DNNs?

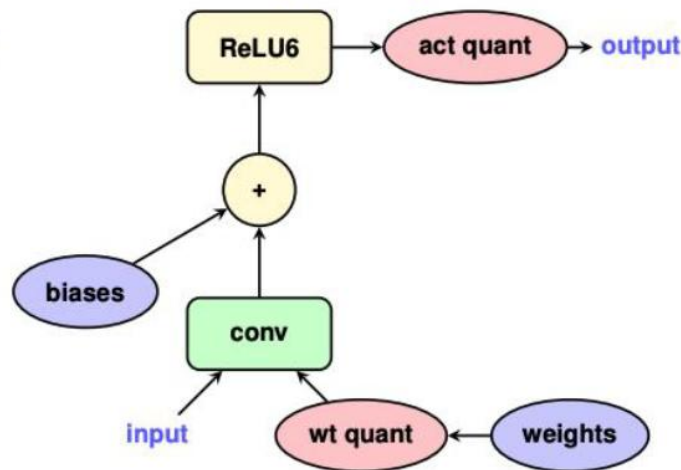


/ Quantization-aware training

- Typically performs better than post-training quantization
- “Simulate” quantization effects in the forward pass
- Weights and biases are updated in floating point during backpropagation so that they can be nudged by small amounts.



(a) Integer-arithmetic-only inference



(b) Training with simulated quantization

/Outlines

- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

/ Unstructured v. s. Structured

Ma XL@ArXiv, 2019

- Non-structured pruning
 - **Pros:** Higher compression rate
 - **Cons:** Only beneficial on dedicated devices
- Structured pruning:
 - **Pros:** Better suitable for more hardware devices
 - **Cons:** Lower compression rate

Could structure and non-structure pruning have similar compression ratio?

	GPU	Multi-core CPU	FPGA	ASIC
Non-Structured Pruning	Not Suitable	Not Suitable	Storage/Computation Dependent ?	
Structured Pruning	Suitable	Suitable		

Is non-structured pruning beneficial at all?

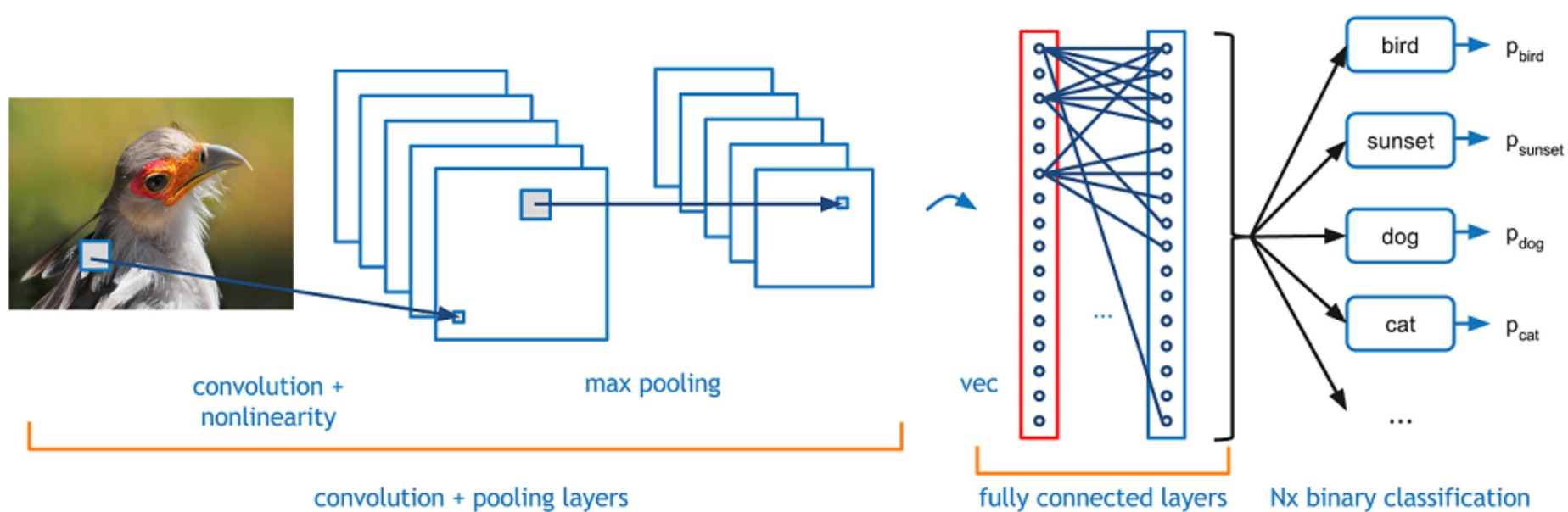
/ Unstructured v. s. Structured

Ma XL@ArXiv, 2019

- How to compare Structured vs Non-Structured Pruning?
 1. Compare with the *same accuracy and SoA practice*.
 2. Compare storage: *weight + indices (absolute/relative)*
 3. Compare *Pruning-to-Performance Ratio (PPR)*.
 - α x weight reduction, β x speedup, $PPR = \alpha/\beta$
 - less PPR -> higher computation efficiency
- For (some) structured pruning, $PPR=1$
- SoA non-structured pruning, $PPR > 2.7$ (Typical ASIC platform)
- Conclusion: non-structured pruning is more beneficial if it can achieve **2.7x more pruning rate** than structured pruning.

/Kernel Operations

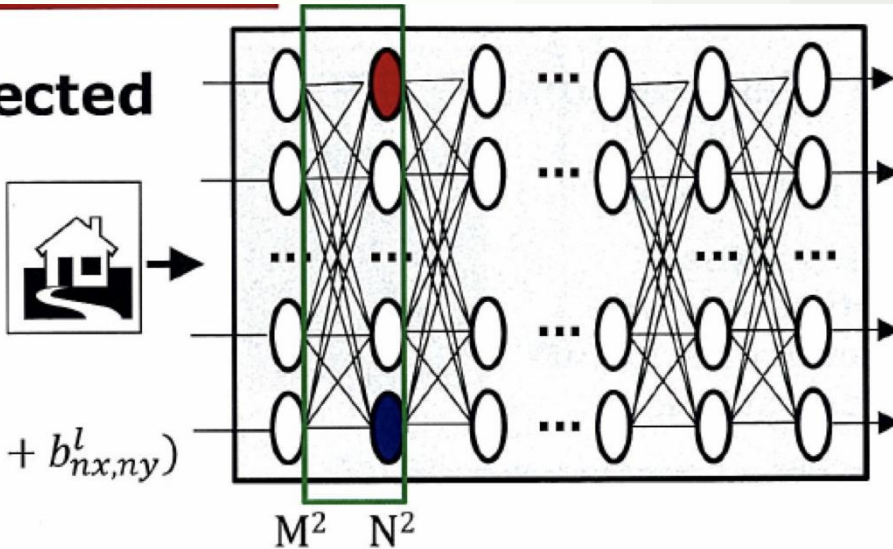
- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**



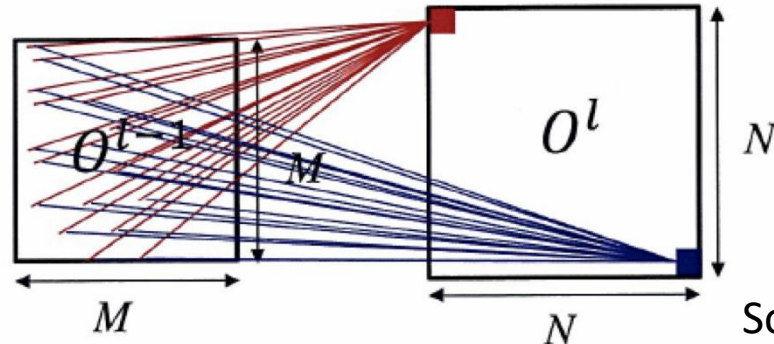
Fully Connected NN

DNN or **fully connected**
(Deep Neural Network)

$$o_{nx,ny}^l = \sigma \left(\sum_{mx,my}^{M,M} w_{mx,my,nx,ny}^l \cdot o_{mx,my}^{l-1} + b_{nx,ny}^l \right)$$



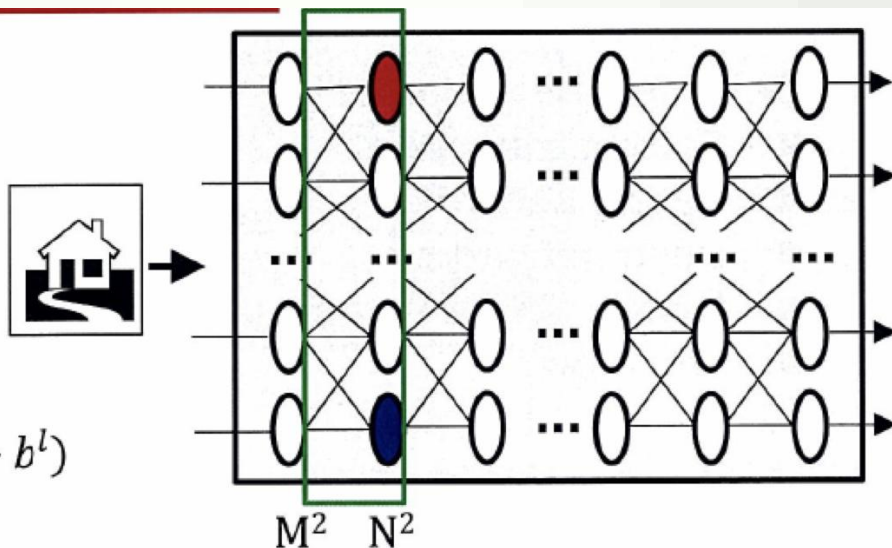
- M and N large!
- $M^2 \times N^2$ weights/layer
- Very flexible
- Memory 🌟, training 🌟



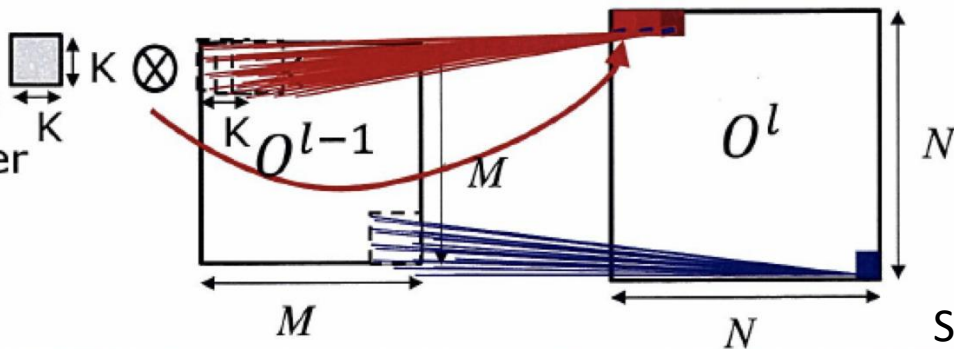
Convolutional NN

CNN or ConvNet (Convolutional Neural Network)

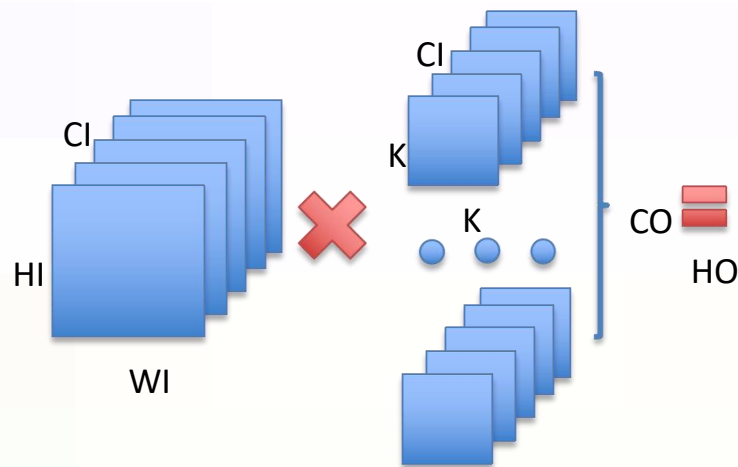
$$o_{nx,ny}^l = \sigma \left(\sum_{kx,ky}^{K,K} w_{kx,ky}^l \cdot o_{xn+kx,ny+ky}^{l-1} + b^l \right)$$



- K^2 weights/layer
- K small ($1 \leq K \leq 11$)
- Less operations per output
- Significantly less weights!

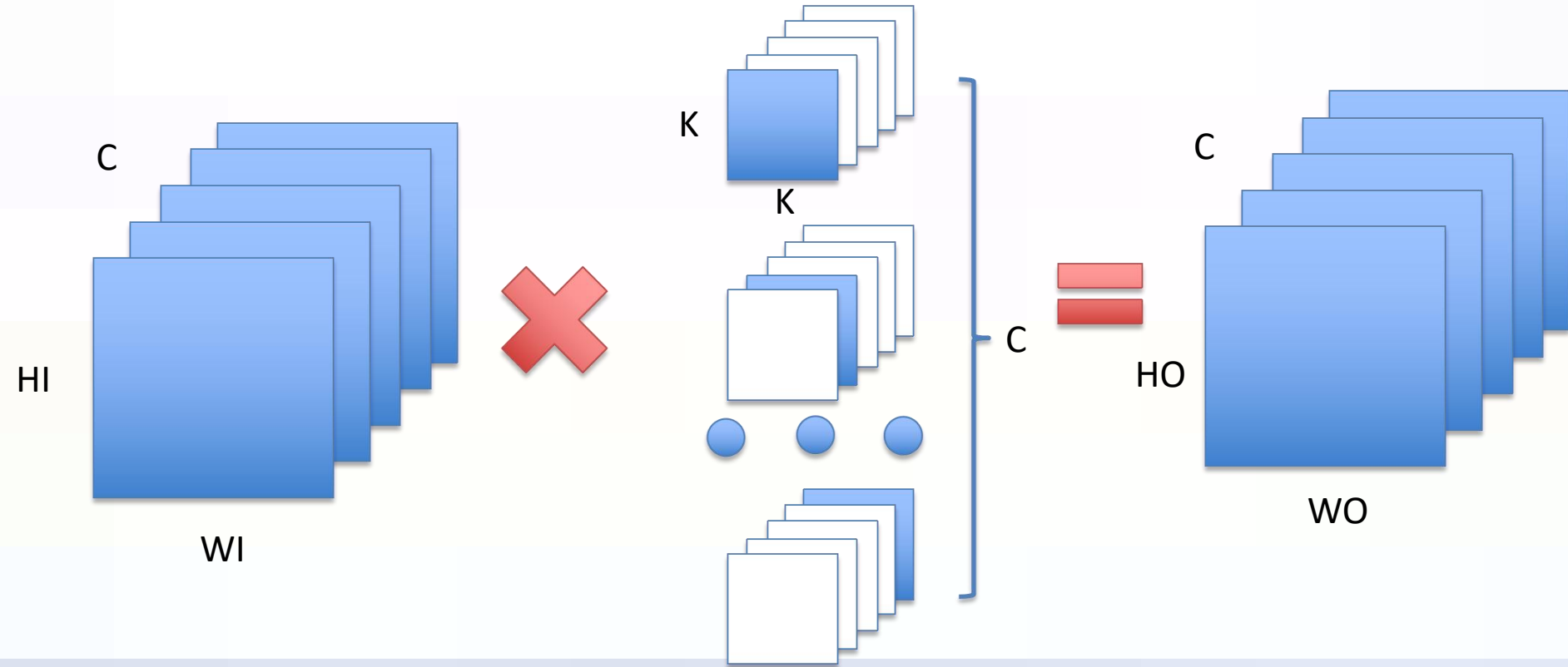


/Convolution Layer

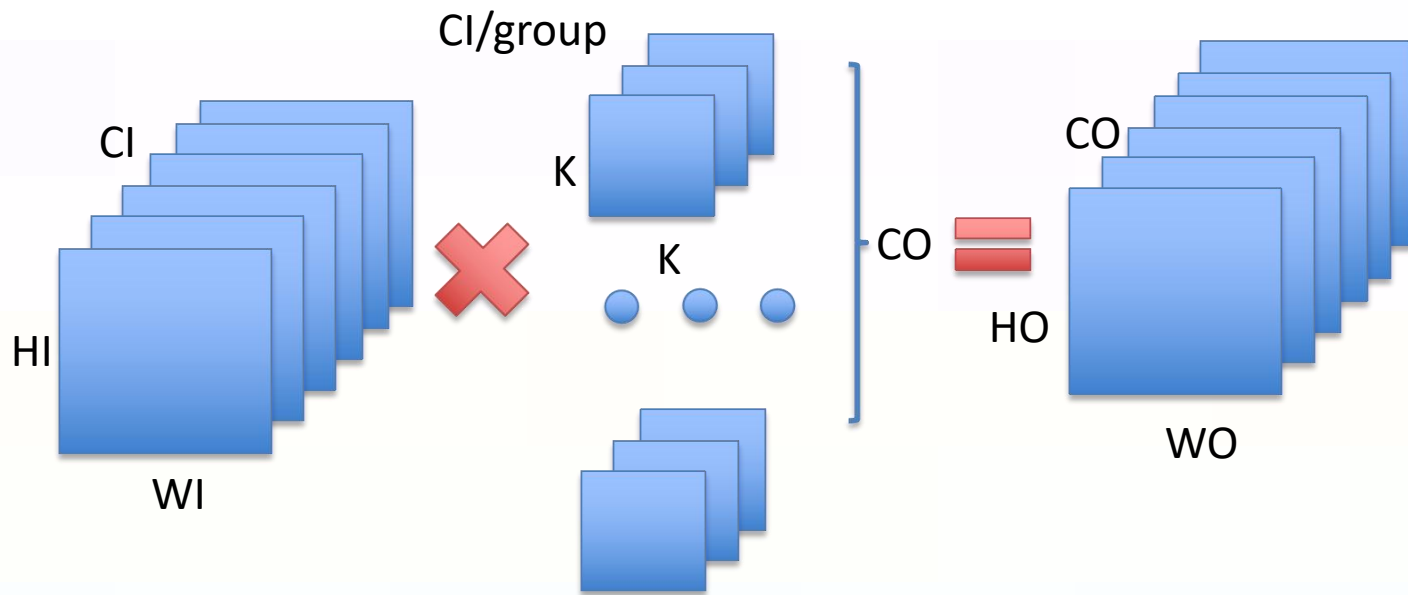


```
for (h=0; h<H_O; h++) {  
    for (w=0; w<W_O; w++) {  
        for (i=0; i<C_O; i++) {  
            OA[i][h][w] = 0;  
            for (k1=0; k1<K; k1++) {  
                for (k2=0; k2<K; k2++) {  
                    for (j=0; j<C_I; j++) {  
                        m = h*stride-pad+k1;  
                        n = w*stride-pad+k2;  
                        OA[i][h][w] +=  
                            IA[j][m][n]*W[i][j][k1][k2];  
                    }  
                }  
            }  
            OA[i][h][w] = Activation(OA[i][h][w])  
        }  
    }  
}
```

/Depth-wise Convolution Layer

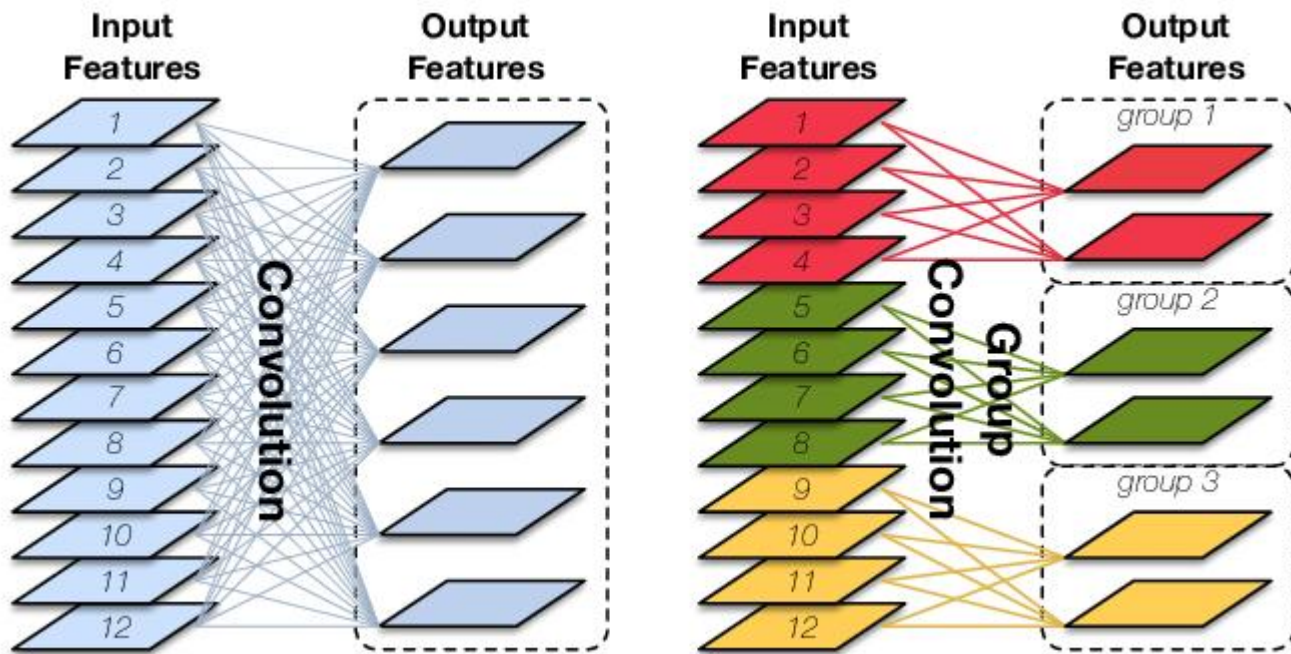


/Group Convolution



Depth-wise convolution是group=CI的特殊情形

/ Group Convolution



Depth-wise convolution是group=CI的特殊情形

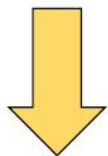
/Transformation for Convolution

- **Direct convolution**
- **GEMM**
- **FFT-based convolution**
- **Winograd convolution**

- **Motivation:** 将卷积操作转换为矩阵乘操作
 - 利用成熟的矩阵运算加速手段
 - 是Caffe等工具采取的神经网络加速手段



Convolution



GEMM

Weight

$$R \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} S$$


Input Activation

$$H \begin{bmatrix} a & b & c \\ d & e & f \\ h & i & j \end{bmatrix} W$$


Output Activation

$$P \begin{bmatrix} A & B \\ C & D \end{bmatrix} Q$$


$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$


$$\begin{bmatrix} a & b & d & e \\ b & c & e & f \\ d & e & h & i \\ e & f & i & j \end{bmatrix}$$


$$\begin{bmatrix} A & B & C & D \end{bmatrix}$$

Winograd convolution

- Motivation: 通过变换减少乘法次数
- Example: $IA=[d_0, d_1, d_2, d_3]$, $conv=[g_0, g_1, g_2]$, 1-D convolution. After $img2col$, we can have

$$OA = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix}$$

Before: 需要六次乘法，四次加法

For Winograd, suppose

$$\begin{aligned} m_1 &= (d_0 - d_2)g_0 & m_2 &= (d_1 + d_2)\frac{g_0 + g_1 + g_2}{2} \\ m_4 &= (d_1 - d_3)g_2 & m_3 &= (d_2 - d_1)\frac{g_0 - g_1 + g_2}{2} \end{aligned}$$

Then we can have

$$OA = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix}$$

After: 需要四次乘法，八次加法

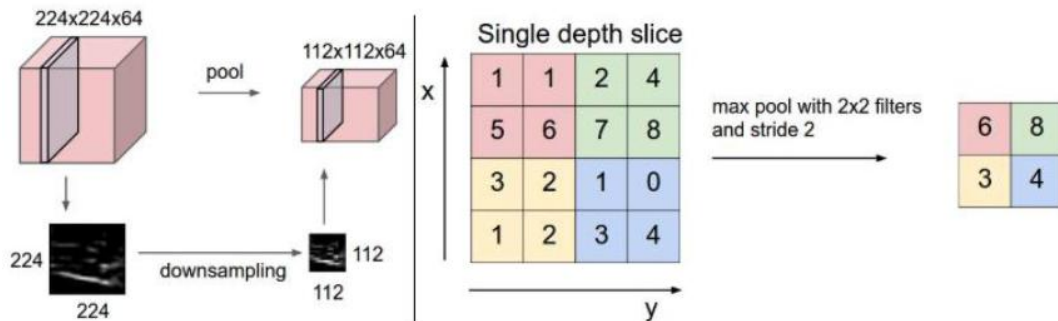
$(g_0+g_1+g_2)/2$ 和 $(g_0-g_1+g_2)/2$ 可以复用

/Outline

- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**

/ Pooling

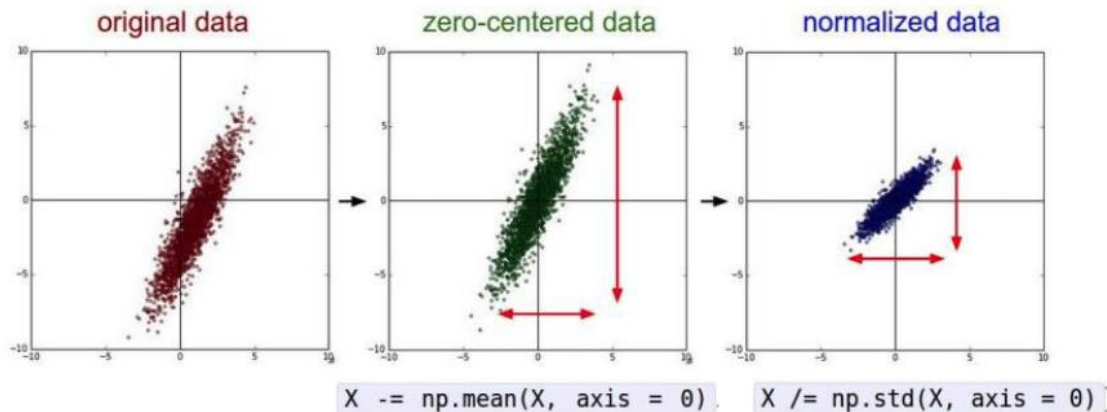
- Progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting.
- Parameters:
 - Type: MAX and AVG
 - Pooling kernel size
 - Pooling stride



Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Left: In this example, the input volume of size $[224 \times 224 \times 64]$ is pooled with filter size 2, stride 2 into output volume of size $[112 \times 112 \times 64]$. Notice that the volume depth is preserved. Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2×2 square).

/ Batchnorm

- Goal: make it easier to train by providing zero-mean, unit-variance activations.
 - The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.



/Batchnorm

Training Time

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$, will recover the
identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

Test Time

$$\mu_j = \text{(Running) average of values seen during training}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \text{(Running) average of values seen during training}$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

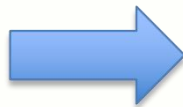
/Batchnorm

- Merge the Batchnorm into convolution during testing



$$y = w * x + b$$

$$z = \frac{y - \text{mean}}{\sqrt{\text{var}}} * \beta + \gamma$$



$$z = w' * x + b'$$

$$w' = \frac{w}{\sqrt{\text{var}}} * \beta$$

$$b' = \frac{b - \text{mean}}{\sqrt{\text{var}}} * \beta + \gamma$$

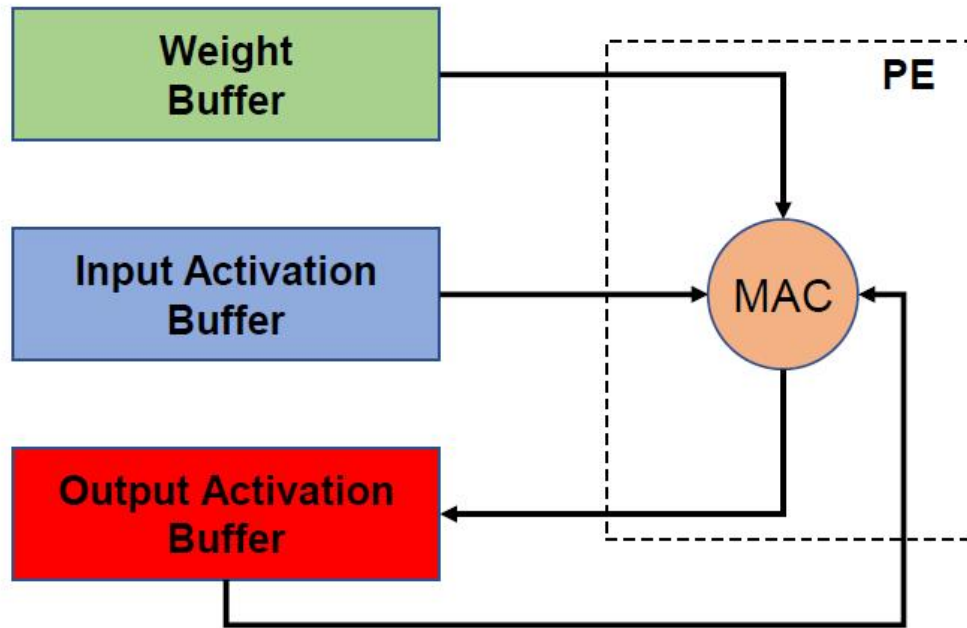
/Outline

- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**

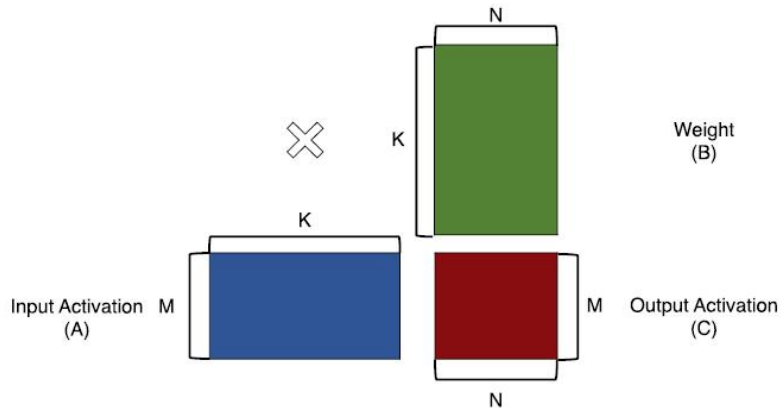
/AI Accelerator

- **Core PE design and weight reuse**
- **Core optimization**
 - **Instruction decoding**
 - **Datapath**
 - **Memory system**
- **CASE study**
 - **Google TPU**
 - **Huawei Ascend (昇腾)**

/ Processing Element (PE) Core Design



/Matrix Multiply



```
for (m=0; m<M; m++) {  
  for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k] * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

for each output activation

convolution window

/ Inst. Decoding Logic Optimization

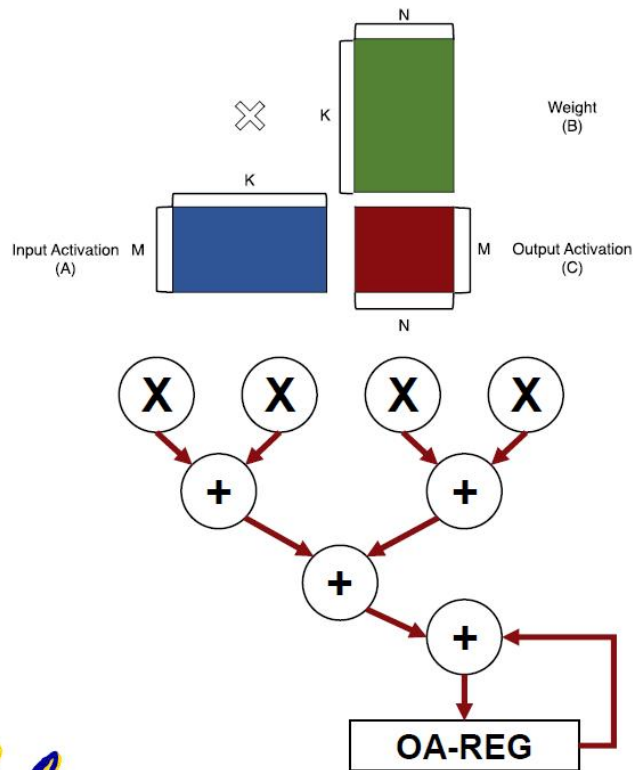
- Coarse-Grained, Domain-Specific Instructions
 - Key instructions: data movement and compute
- Example: TPU ISAs

```
10000170: 100c0993      + addi+ x19,x24,-256
10000174: 00004a97      + auipc+ x21,0x4
10000178: 674a8a93      + addi+ x21,x21,1652 # 100047e8 <8>
1000017c: 000a8913      + mv+ x18,x21
10000180: 000a8413      + mv+ x8,x23
10000184: 00000493      + li+ x9,0
10000188: 00022583      + lw+ x11,0(x18)
1000018c: 00042503      + lw+ x10,0(x8)
10000190: 00440413      + addi+ x8,x8,4
10000194: 10030913      + addi+ x18,x18,256
10000198: f25ff0ef      + jal+ x1,1000000c <times>
1000019c: 00a484b3      + add+ x9,x9,x10
100001a0: 0093a823      + sw+ x9,0(x19)
100001a4: ff4412e3      + bne+ x8,x20,10000188 <mult+0x6c>
100001a8: 00498993      + addi+ x19,x19,4
100001ac: 009b0b33      + add+ x22,x22,x9
100001b0: 004a8b93      + addi+ x21,x21,4
100001b4: fd8994e3      + bne+ x19,x24,1000017c <mult+0x60>
100001b8: 10040a13      + addi+ x20,x8,256
100001bc: 10098c13      + addi+ x24,x19,256
100001c0: 100b8b93      + addi+ x23,x23,256
100001c4: fb9a16e3      + bne+ x20,x25,10000170 <mult+0x54>
```

1. Read_Host_Memory reads data from the CPU host memory into the Unified Buffer (UB).
2. Read_Weights reads weights from Weight Memory into the Weight FIFO as input to the Matrix Unit.
3. MatrixMultiply/Convolve causes the Matrix Unit to perform a matrix multiply or a convolution from the Unified Buffer into the Accumulators. A matrix operation takes a variable-sized B*256 input, multiplies it by a 256x256 constant weight input, and produces a B*256 output, taking B pipelined cycles to complete.
4. Activate performs the nonlinear function of the artificial neuron, with options for ReLU, Sigmoid, and so on. Its inputs are the Accumulators, and its output is the Unified Buffer. It can also perform the pooling operations needed for convolutions using the dedicated hardware on the die, as it is connected to nonlinear function logic.
5. Write_Host_Memory writes data from the Unified Buffer into the CPU host memory.



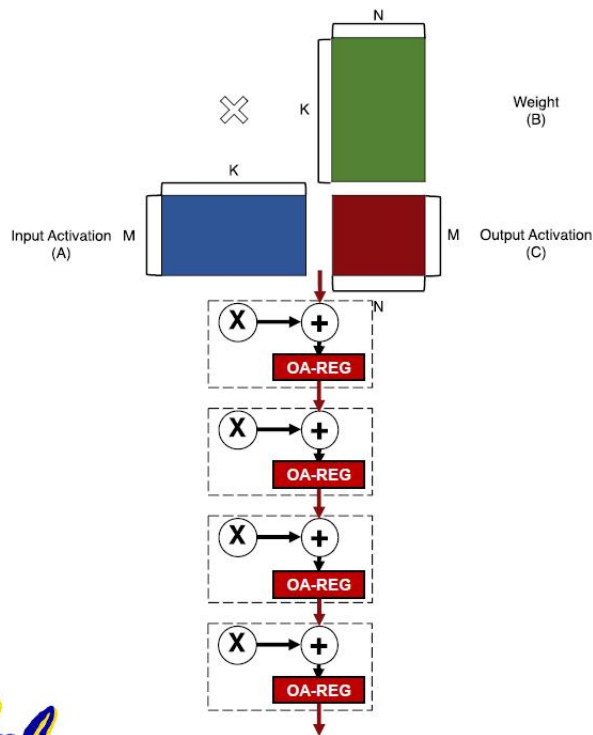
/ Datapath Optimization 1: Spatial-K



```
for (m=0; m<M; m++) {  
  for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    spatial_for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
                * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

- Type 1: Adder Tree
- Example: NVDLA, DianNao
- Typical width: 8-64
- Applicable to any accumulation dimensions
 - E.g., R, S, C in convolution

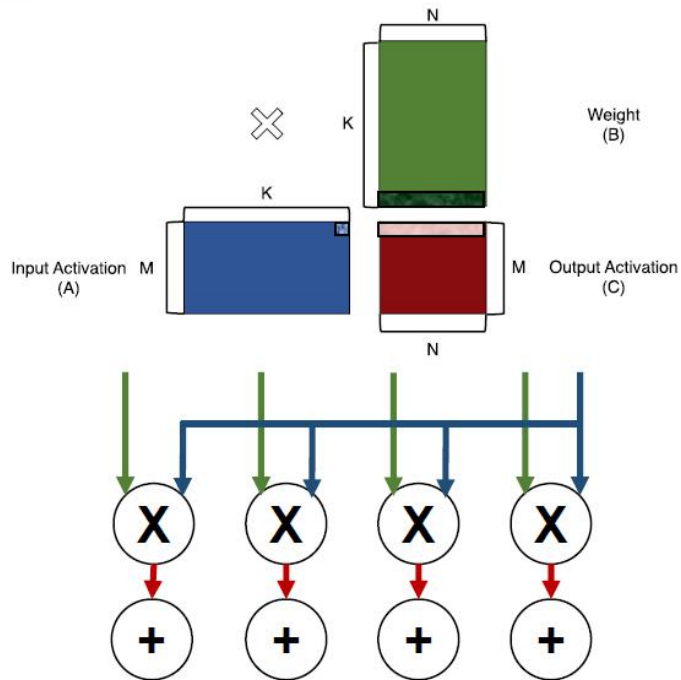
/ Datapath Optimization 1: Spatial-K



```
for (m=0; m<M; m++) {  
  for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    spatial_for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
                * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

- Type 2: Systolic Accumulation
- Example: TPU, Gemini
- Typical width: 8-256
- Applicable to any accumulation dimensions
 - E.g., R, S, C in convolution

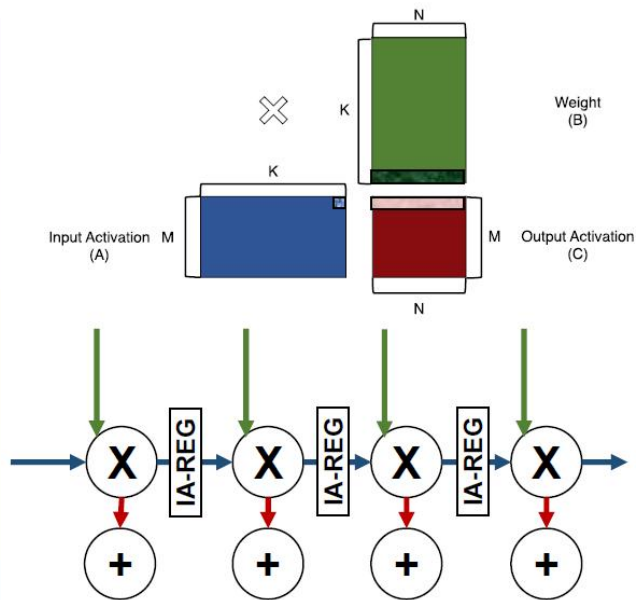
/ Datapath Optimization 2: Spatial-N



```
for (m=0; m<M; m++) {  
    spatial_for (n=0; n<N; n++) {  
        OA[n,m] = 0;  
        for (k=0; k<K; k++) {  
            OA[n,m] += IA[m, k]  
                      * W[k, n];  
        }  
        OA[n,m] = Activation(OA[n,m]);  
    }  
}
```

- Type 1: Direct-wiring multicast
- Example: NVDLA, DianNao
- Typical width: 8-16
- Applicable to any non-accumulation dimensions

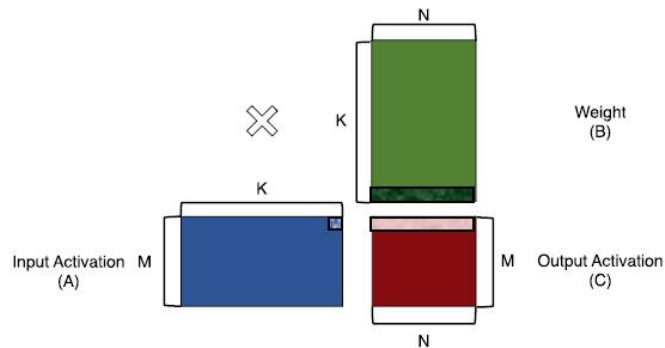
/ Datapath Optimization 2: Spatial-N



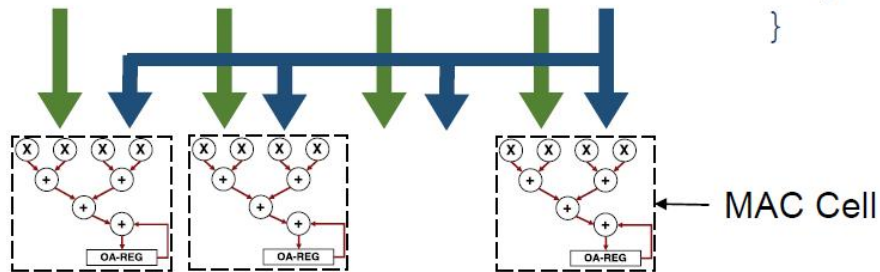
```
for (m=0; m<M; m++) {  
  spatial_for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
                * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

- Type 2: Systolic multicast
- Example: TPU, Gemmini
- Typical width: 8-256
- Applicable to any non-accumulation dimensions

/ Datapath Optimization Combined: NVDLA

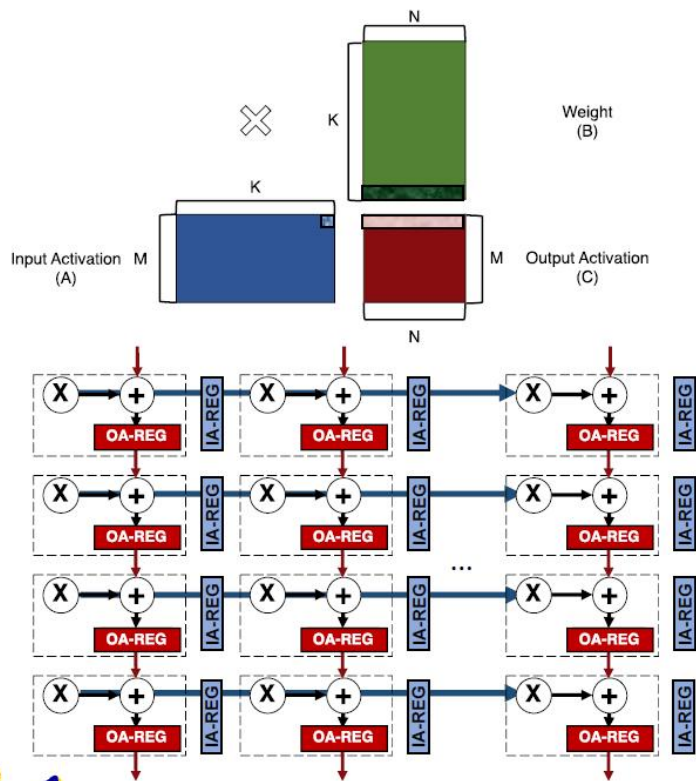


```
for (m=0; m<M; m++) {  
  spatial_for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    spatial_for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
        * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```



- Adder-tree accumulation
- Direct-wiring multicast

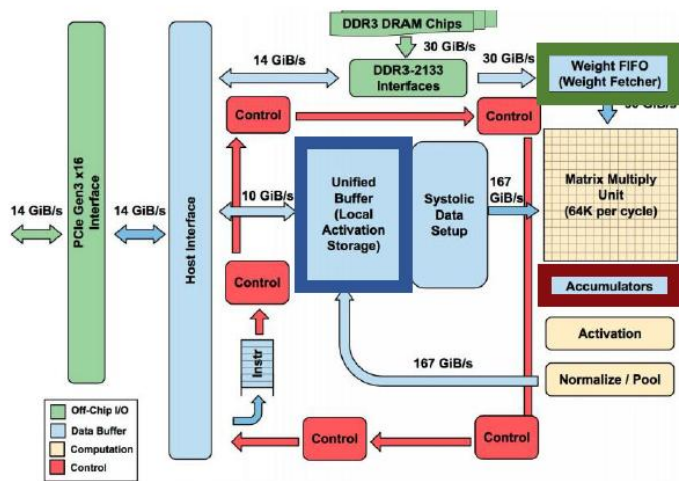
/ Datapath Optimization Combined: TPU



```
for (m=0; m<M; m++) {  
  spatial_for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    spatial_for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
        * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

- Systolic accumulation
- Systolic multicast

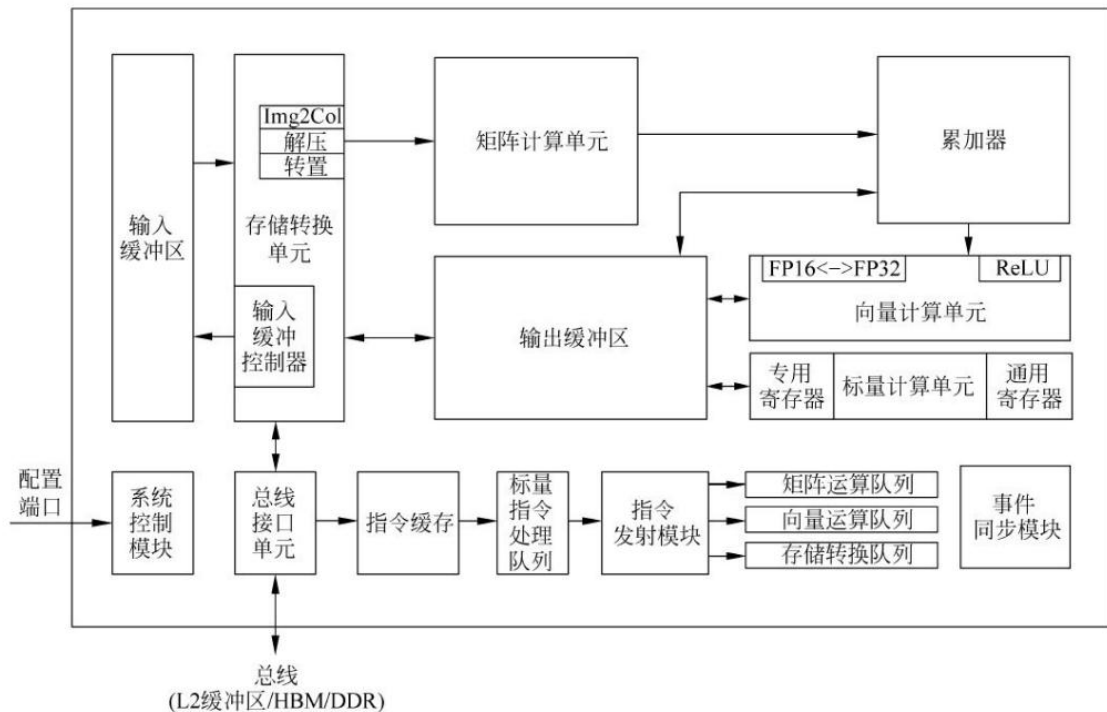
/Tensor Processing Unit



TPU

- Inst. Decoding Logic:
 - Coarse-grained matrix multiply and data read/write instructions
- Datapath:
 - Spatial-K: Systolic Accumulation
 - Multi-cycle with registers
 - Spatial-N: Systolic multicast
 - Multi-cycle with registers
 - Better scalability
- Memory
 - Custom systolic registers
 - Dedicated accumulation and weight buffers
 - Double-buffered, weight-stationary dataflow

AI Core



三种计算资源：矩阵计算单元，向量计算单元，标量计算单元，三条独立流水线设计，对应三个指令发射序列。

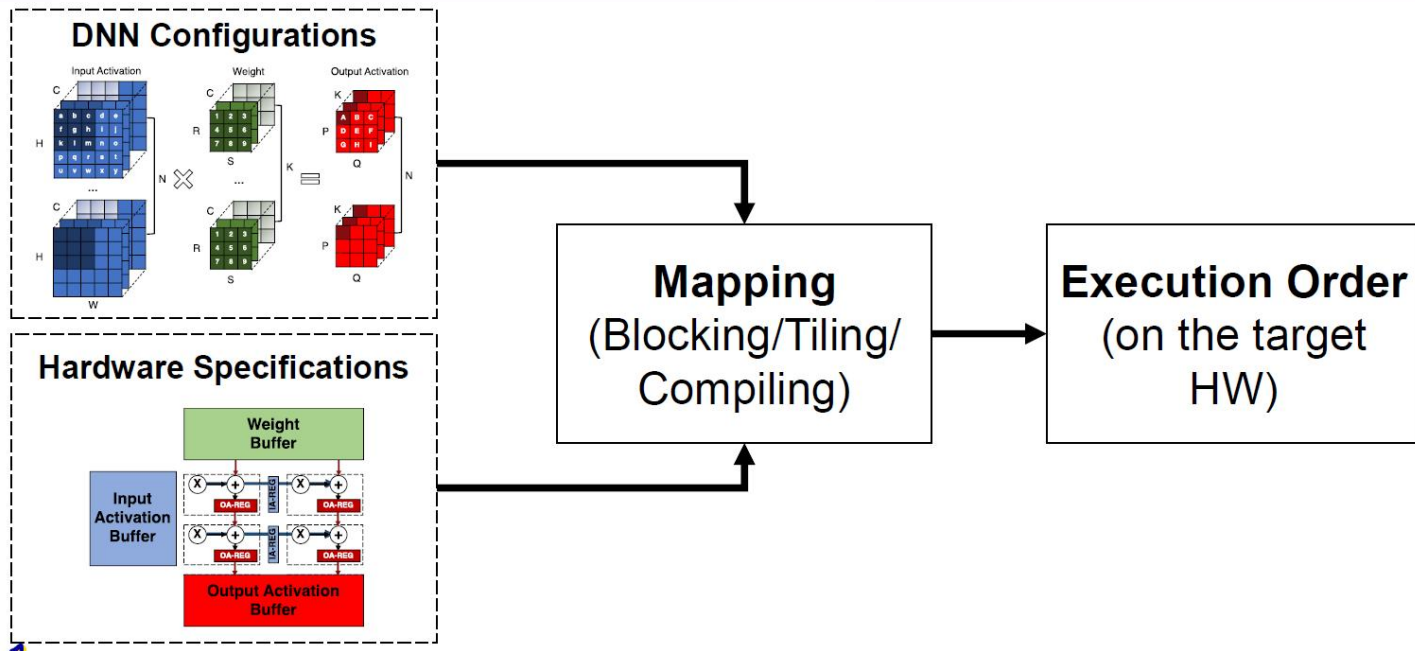
存储转换单元：硬件完成Img2Col将输入网络和特征值重排进行输入。对比GPU使用软件实现效率更高。

/AI ToolChain

- **DNN Mapping Problem**
 - **Loop Nest**
 - **HW Constraints**
- **Mapping Space**
 - **Loop Ordering**
 - **Loop Bound**
 - **Spatial Choice**
- **Toolchain**

/Review: The DNN Mapping Problem

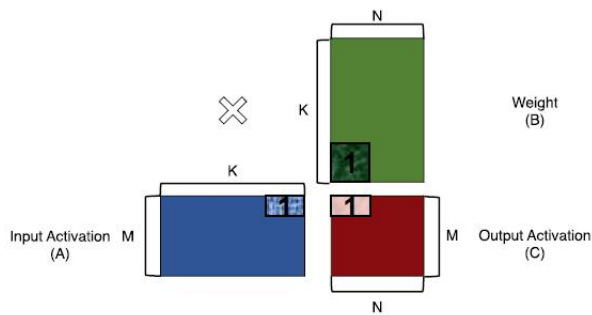
Mapping: Large network model -> limited hardware resources.
Reducing additional data movement/recompute overheads.



/Q1: What if the systolic array size $< K$ or N ?

Can not mapping the all weights to the systolic array in the same time.

Adding two bounds, i.e., K_1 and N_1 , to mapping part of the weights.

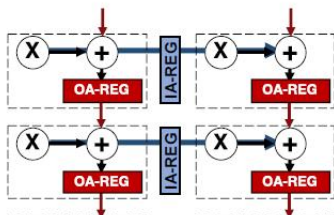


- Hardware Specifications:

- Systolic array size: 2×2

- Notation:

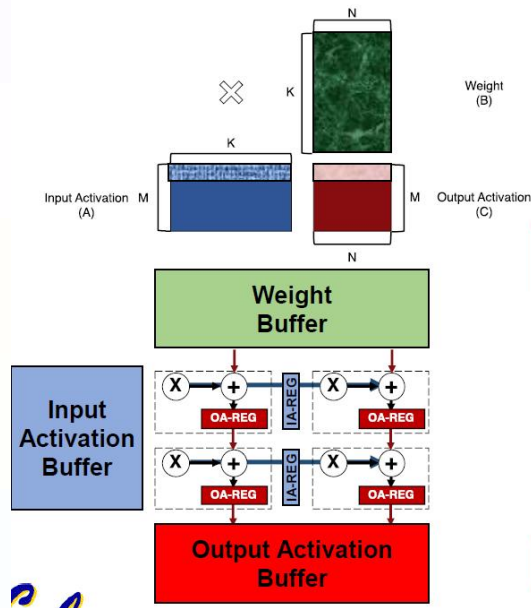
- K_1/N_1 : loop bounds



```
for (m=0; m<M; m++) {  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
}
```


/Q2: What if weight buffer size $< N \times K$?

Weights need to be move in/out to the weight buffer. Optimize to minimize data movements.

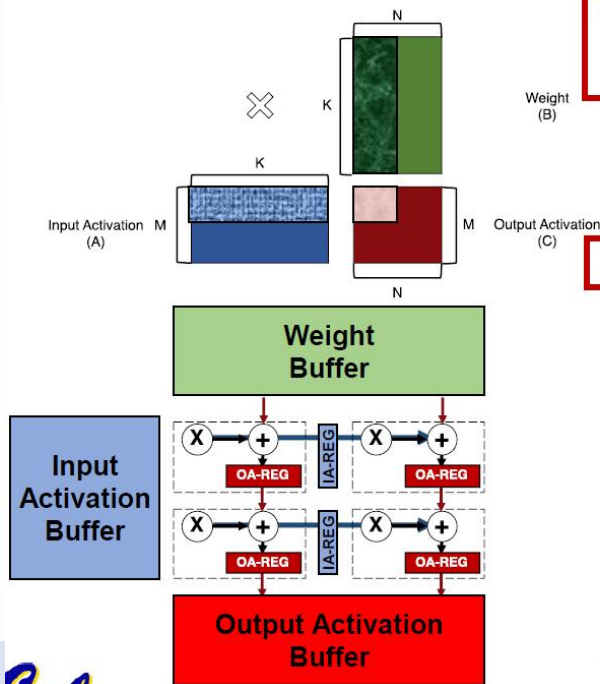


- Hardware Specifications:
 - Systolic array size: 2×2
 - Explicit data movement
 - Output/Weight/Input buffer sizes

```
for (m=0; m<M; m++) {  
  mvin(W[0:K,0:N]); // W buffer  $\geq N \times K$   
  mvin(IA[m:m+1,0:K]); // IA buffer  $\geq 1 \times K$   
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
  mvout(OA[0:N,m:m+1]); // OA buffer  $\geq 1 \times N$   
}
```


/Q3: What if input buffer size $> 1 \times K$?

Should store > 1 rows of input activations.



```

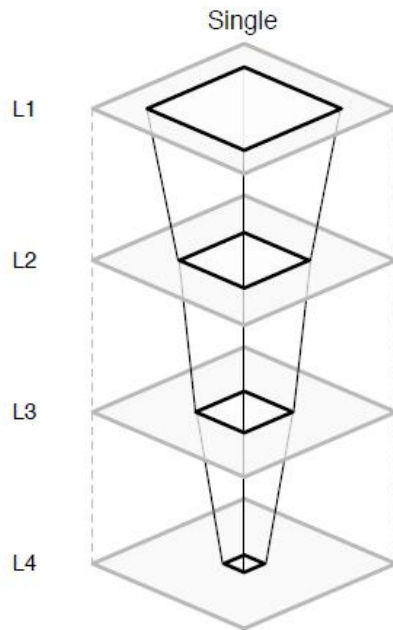
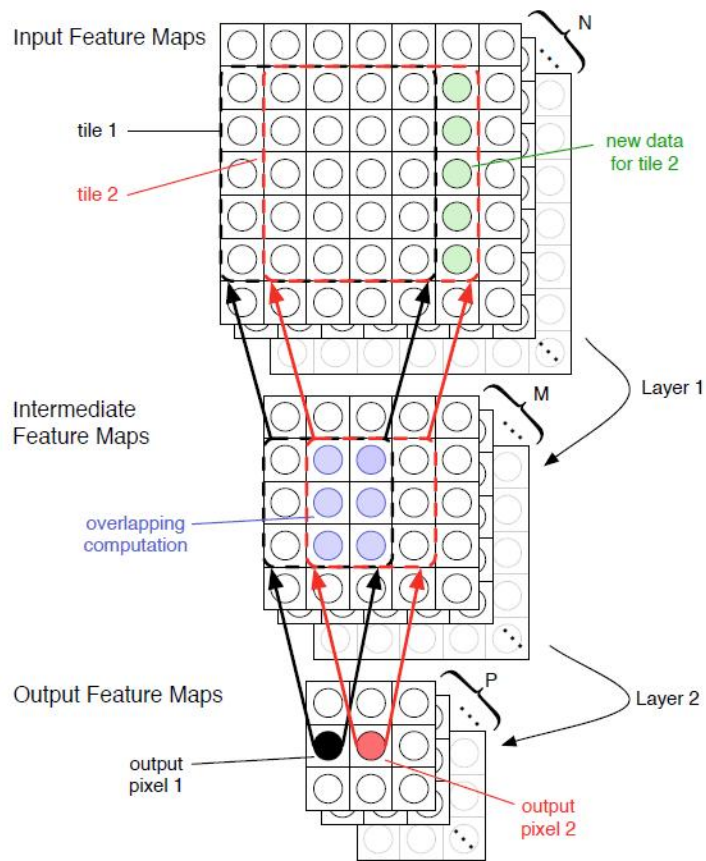
for (m2=0; m2<M2; m2++) {
    // IA buffer stores: M1*K
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1] = 0;
        for (m1=0; m1<M1; m1++) {
            for (n1=0; n1<N1; n1++) {
                for (k1=0; k1<K1; k1++) {
                    spatial_for (n0=0; n0<N0; n0++) {
                        spatial_for (k0=0; k0<K0; k0++) {
                            OA[n2*N1*N0+n1*N0+n0, m2*M1+m1]
                                += IA[m2*M1+m1, k1*K1+k0]
                                    * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];
                        }
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1]);
    }
}

```

/ Navigating the Mapping Space

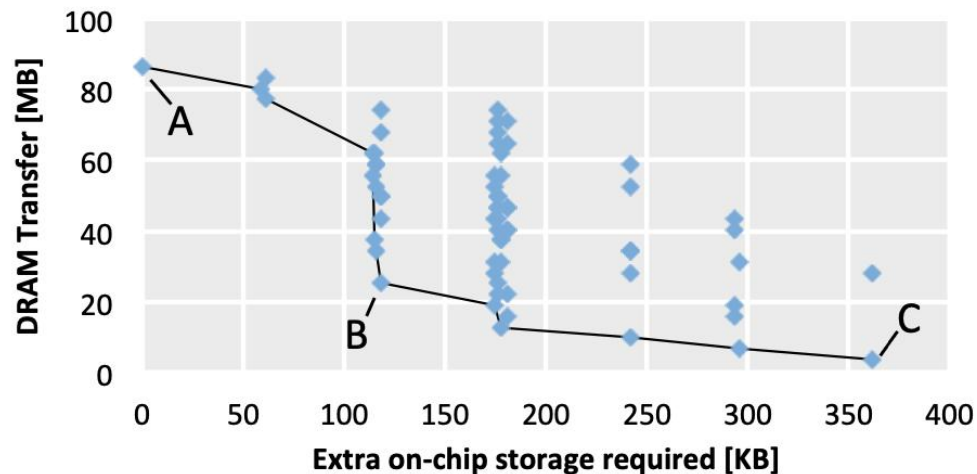
- DNN mapping problem can be viewed as an optimization problem.
- Given:
 - DNN dimensions (N, H, W, C, R, S, K, stride, padding)
 - Hardware specifications (dataflow, memory hierarchy)
- Find:
 - An optimal loop nest that minimizes latency and/or energy.
 - That defines both temporal and spatial execution order
- Approaches:
 - Exhaustive search
 - Random search
 - Learning-based algorithms
 - Optimization-based algorithms

/ Layer Fusion Concept



- 多层数据同时缓存于片上
- **金字塔式输入特征图**
- 增量式瓦片 (tile) 数据
- 重叠计算保持片上数据

/Memory Access under Various Fusion Size

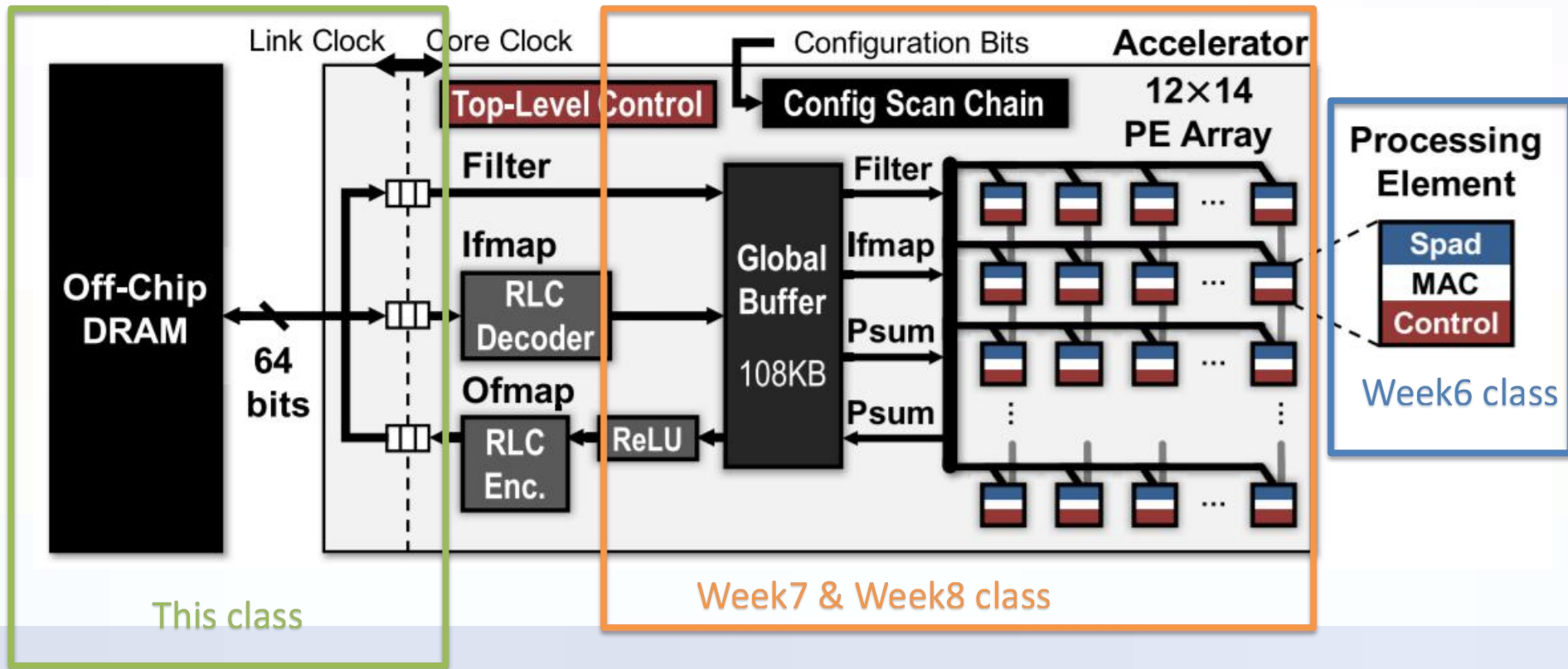


(b) VGGNet-E

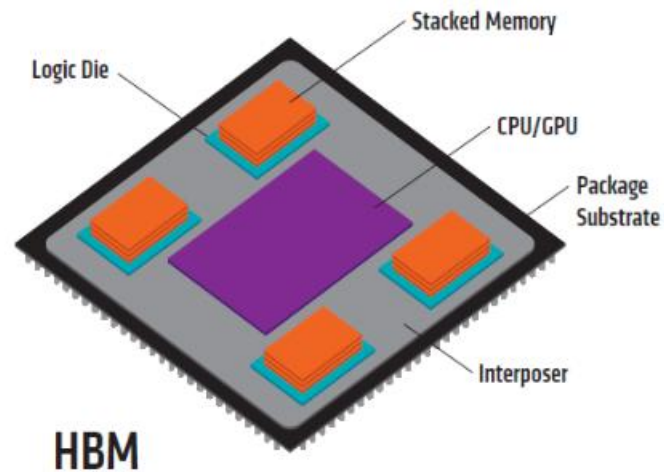
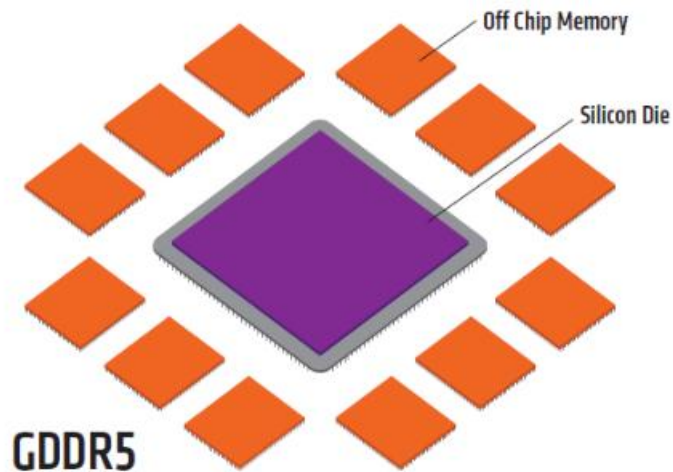
- 邻近原点数据点具有更好性能
- A: No fusion, 86MB DRAM
- C: All fusion, 3.6MB DRAM

	Fused-Layer	Baseline
MB transferred/input	3.64	77.14
Cycles $\times 10^3$	11,665	10,951
BRAMs	2,509	2,085
DSP48E1	2,987	2,880

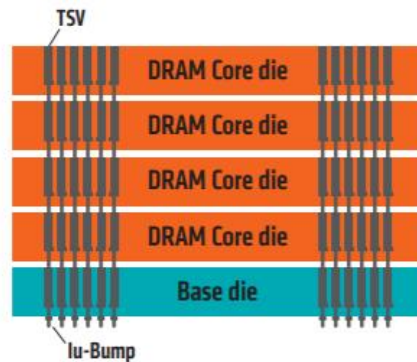
/Review: A Typical AI Accelerator



HBM vs GDDR5: HBM shortens your information commute



HBM vs GDDR5: Compare side by side



GDDR5	Per Package	HBM
32-bit	Bus Width	1024-bit
Up to 1750MHz (7GBps)	Clock Speed	Up to 500MHz (1GBps)
Up to 28GB/s per chip	Bandwidth	>100GB/s per stack
1.5V	Voltage	1.3V

Sparsity-Aware Tutorial

- ❑ Introduction to Neural Network Sparsity
- ❑ **Spatial-Domain Sparsity Based NN Processor**
- ❑ Temporal-Domain Sparsity Based NN Processor
- ❑ Sparsity-Aware CIM NN Processor
- ❑ Conclusion

Challenge 1: Wide Sparsity Distribution

➤ Dimension 1: Different Network / Data Type

Weights

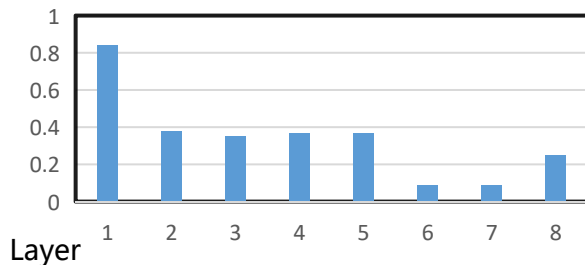
Network	Sparsity
LeNet-5	8%—66%
AlexNet	9%—84%
VGG	4%—58%

Activations

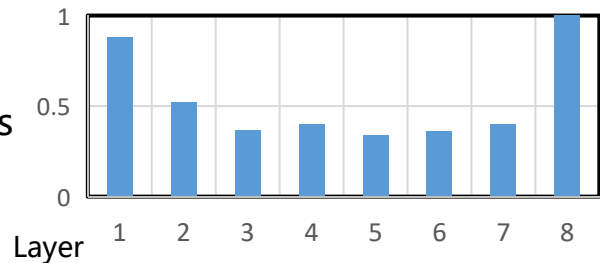
Network	Sparsity
LeNet-5	55%—82%
AlexNet	34%—88%
VGG	19%—89%

➤ Dimension 2: Different Layers in a Network

Alexnet
Weights
Sparsity



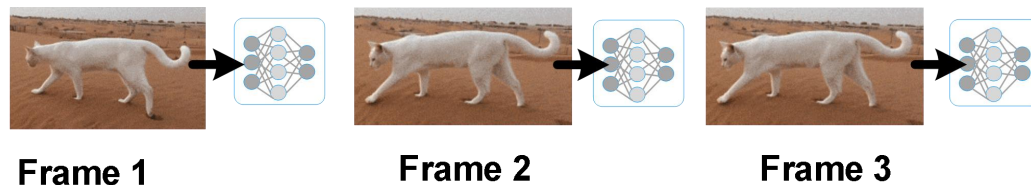
Alexnet
Activations
Sparsity



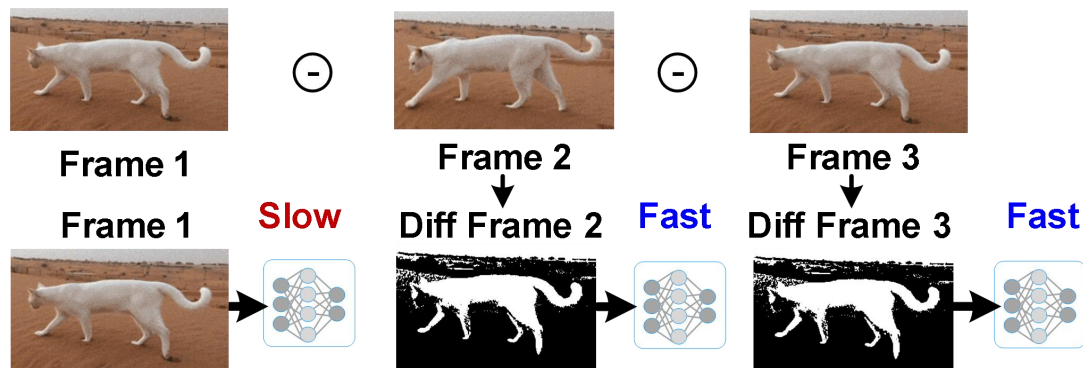
➤ Efficiently Process **All Sparse Situations** Is Important!

Differential Frame CNN

Traditional Way
Process Full Frame



New Dimension
Process Differential Frame



Hybrid-Precision Diff-Frame CNN

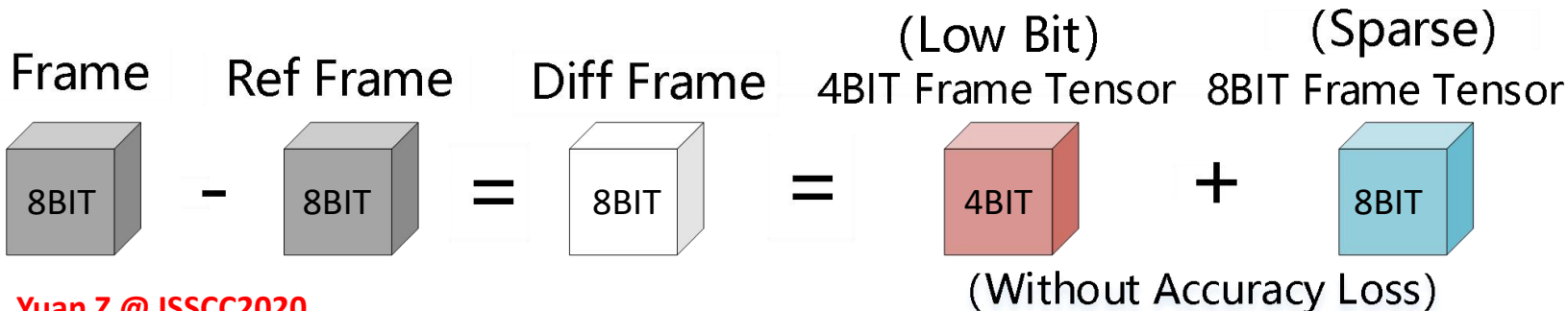
- Use Inter Frame Similarity
 - Get Diff Frame First
 - Diff Frames are not Sparse
- Hybrid-Precision
 - A Low-Bit Tensor
 - A High-Bit Sparse Tensor
 - No Accuracy Loss



Original Frame



Diff Frame



/Computing-in-Memory

- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
 - **Challenges and Trade-offs in CIM**

/ Typical Components in CIM

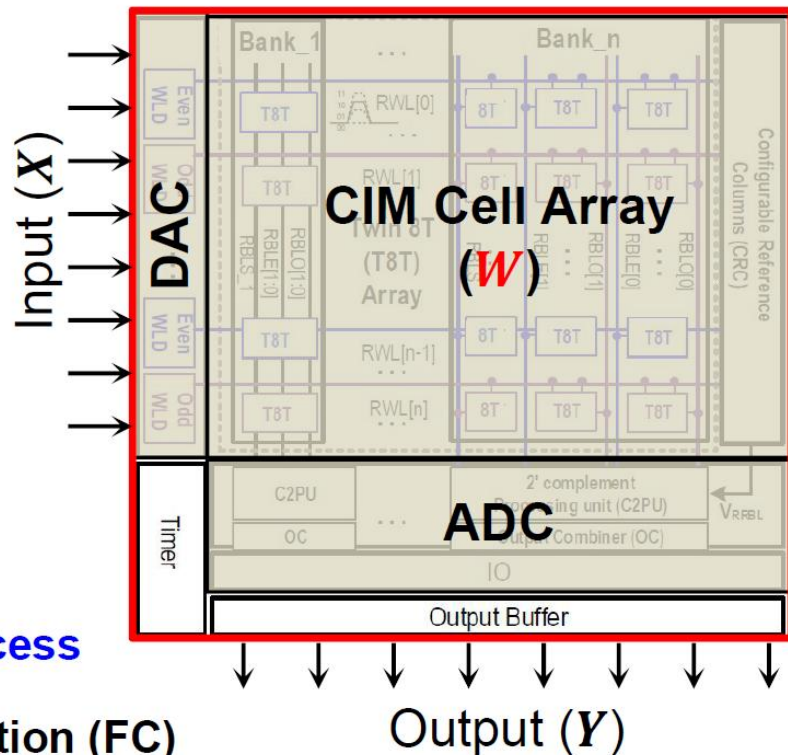
Main Modules: DAC, ADC, CIM Array

$$Y = W \cdot X$$

$$\begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_N \end{bmatrix}^T \cdot \begin{bmatrix} W_{11}, \dots, W_{1M} \\ W_{21}, \dots, W_{2M} \\ \dots \\ W_{N1}, \dots, W_{NM} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_N \end{bmatrix}$$

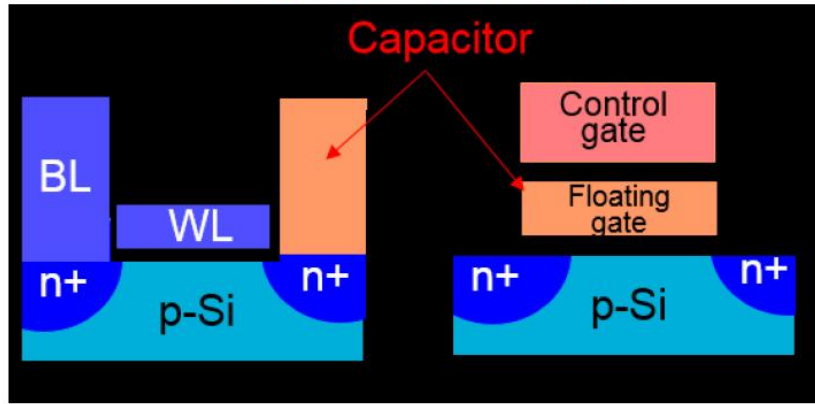
Large W , small X/Y to minimize data access

Ideal application: matrix-vector multiplication (FC)



/Constituent Elements

Charge-based memory



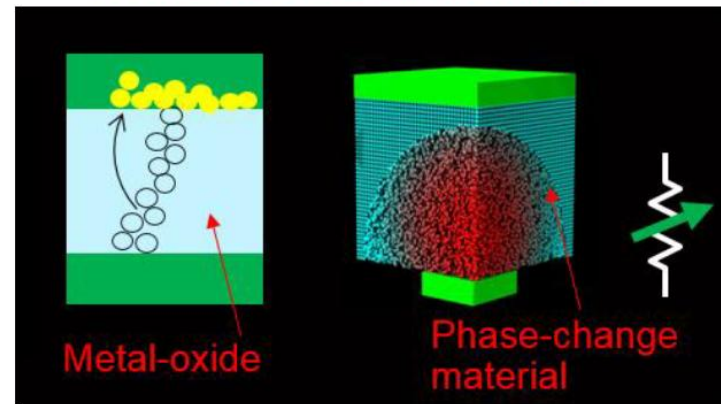
Coulomb's law

Aga et al., HPCA (2017) (SRAM)

Seshadri et al., MICRO (2017) (DRAM)

Merrikh-Bayat, IEEE TNNLS (2018) (Flash)

Resistance-based memory



Ohm's law

Burr et al., Adv. Phys. X (2017)

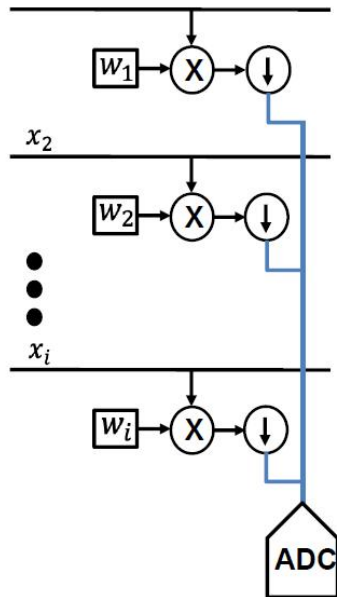
Sebastian et al., J. Appl. Phys. (2018)

Ielmini and Wong, Nature Electr. (2018)

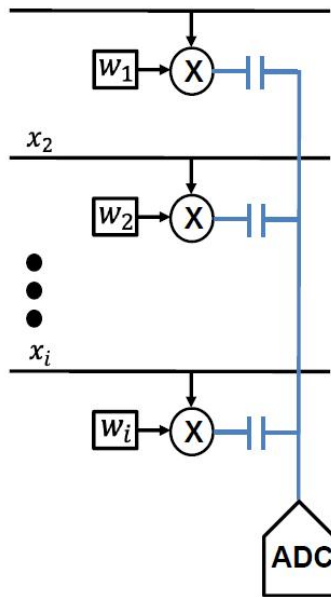
- **Charge-based memory:** Presence or absence of charge (eg. DRAM, SRAM, Flash)
- **Resistance-based memory:** Differences in atomic arrangements or orientation of ferromagnetic metal layers (eg. PCM, metal-oxide RRAM, STT-MRAM)
- Several **computational primitives realized by both types of memory**

Three Types of SRAM-based CIM

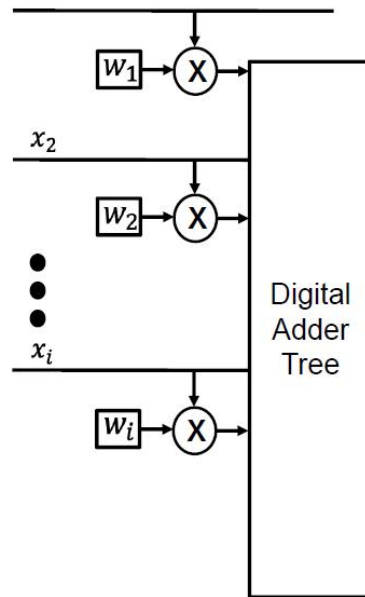
Classified by: Result generation format on each single CIM cell



Current-based CIM
(Less transistors)

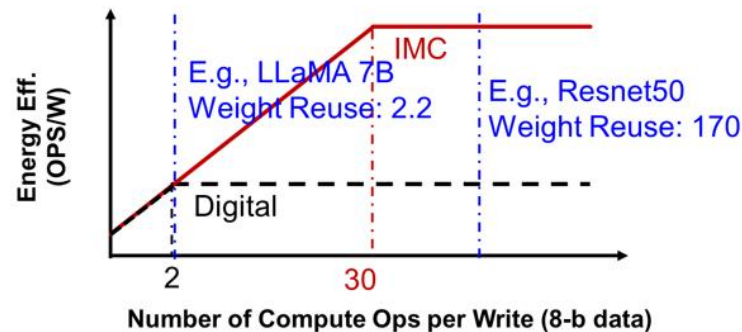
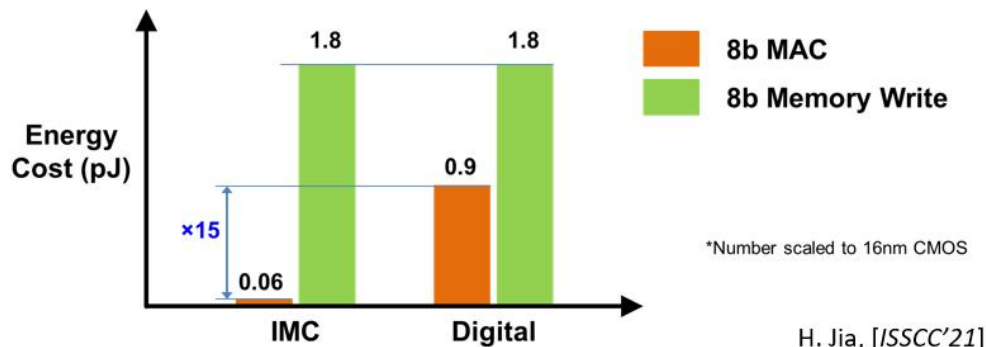


Charge-based CIM
(High parallelism)



All-digital CIM
(No accuracy loss)

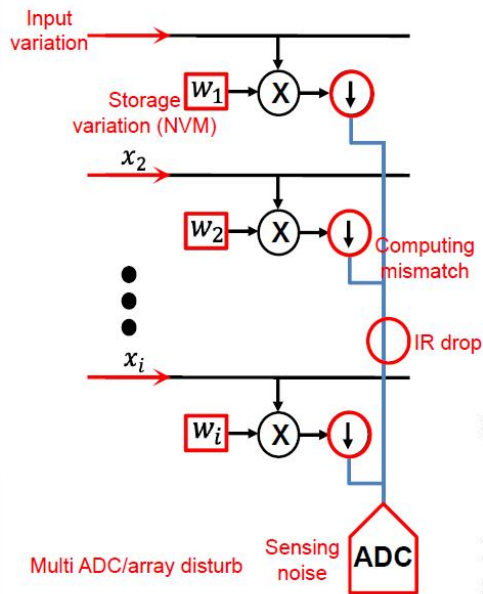
/Challenges of scalable IMC



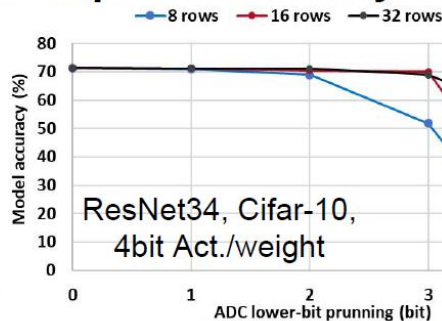
- IMC does not help memory write (NN weight loading)
- weight-writing costs must be adequately amortized (i.e. weight reuse)

Accuracy Trade-Off in CIM

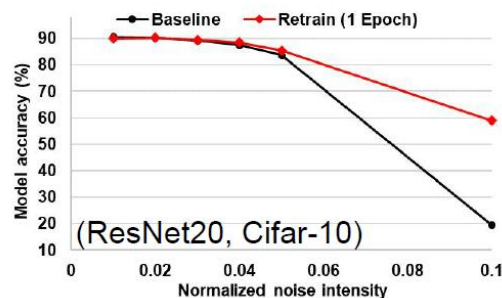
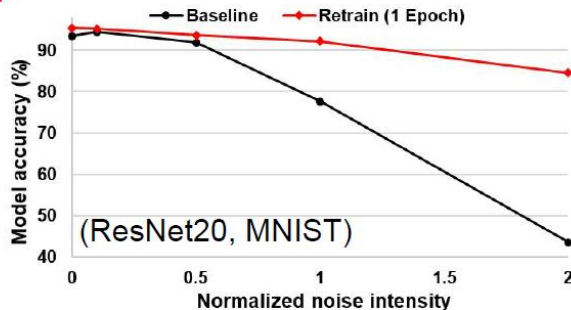
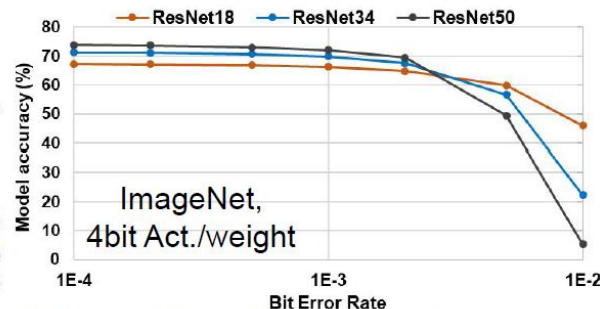
- Parallelism & ADC precision for power/accuracy trade-off
- Re-train is effective to improve accuracy



Variation in CIM circuits



Accuracy vs. parallelism/ADC precision/Network depth



/ End2End Deploy & ALP

- **End-to-End Deployment**

- **AI Processing Pipeline**

- **Practical Constraints:**

- Thermal, Performance Variability, Interference**

- **Accelerator-Level Parallelism (ALP)**

- **X-level Parallelism**

- **ALP in Mobile SoC**

- **Open Challenges**

/ Holistic View of the ML Processing Pipeline

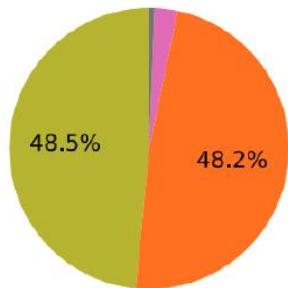
- End-to-end ML pipeline is more than the ML algorithms.
 - ML algorithms also rely on pre- and post-processing code typically executed on CPUs
- For example:
 - “Data processing machines, aka ‘readers’, read the data from storage, process and condense them, and then send to the training machines.”
 - “Training machines, aka ‘trainers’, solely focus on executing the training options rapidly and efficiently. ”

*Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective,
HPCA'2018*



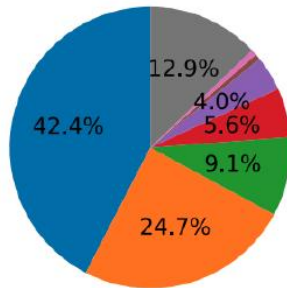
/ Example: A Face Recognition Pipeline

- CPU time breakdown for different components.
- With Amdahl's law, 8X AI speedup $\rightarrow$ 1.59X detection speedup.



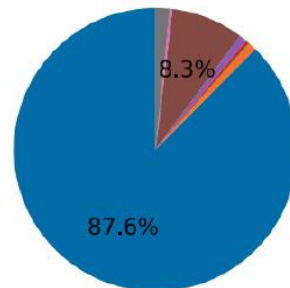
■ Frame Extraction
■ Frame Resize
■ Logging
■ Other

(a) Ingestion.



■ AI
■ Face Resize
■ Networking
■ Other Tensorflow
■ Numpy
■ Kafka
■ Logging
■ Other

(b) Detection.



■ AI
■ Image Mgmt
■ Networking
■ Other Tensorflow
■ Numpy
■ Kafka
■ Logging
■ Other

(c) Identification.

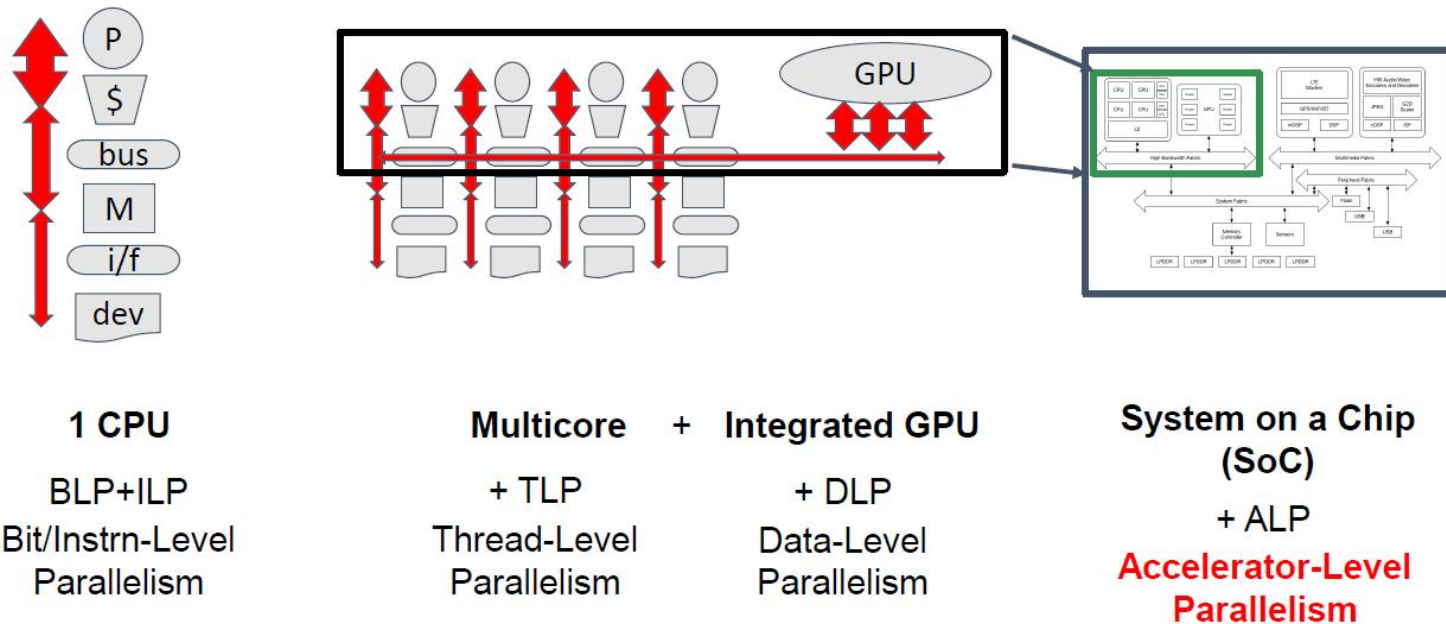
*Missing the Forest for the Trees:
End-to-End AI Application
Performance in Edge Data
Centers, HPCA'2020*

/Outline

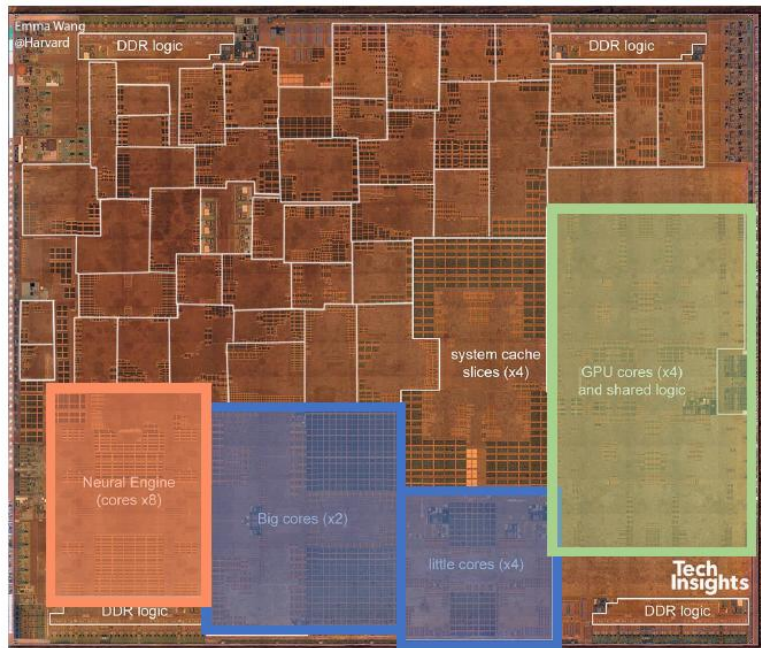
- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/X-Level Parallelism in Computer Architecture

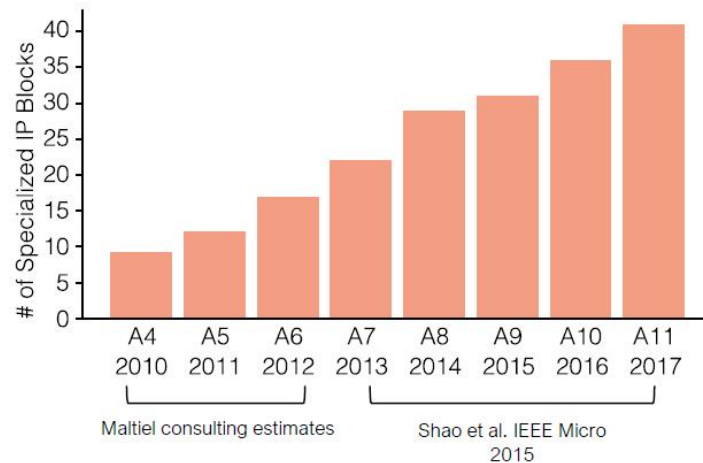
- Accelerator-level parallelism (ALP): the parallelism of different workload components concurrently executing on multiple accelerators.



/CPU, GPU, and Accelerators



Apple A12 SoC



<http://vlsiarch.eecs.harvard.edu/research/accelerators/die-photo-analysis/>

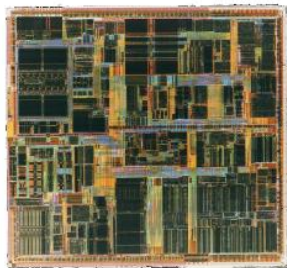
/Outline

- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/X-level Parallel Hardware + Software (Model)

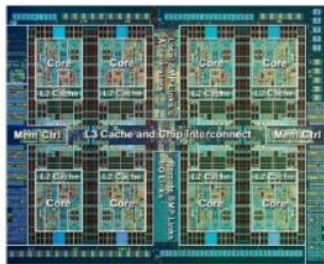
Single-threaded
languages,
compilers,
runtimes, etc.

hides parallelism



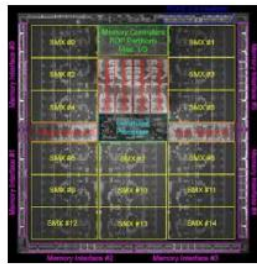
Intel Pentium Pro
BLP+ILP

Software
abstracts TLP,
e.g., pThreads,
OpenMP, & MPI.
Also **hidden** in
cloud, etc.



IBM Power 7
BLP+ILP+TLP

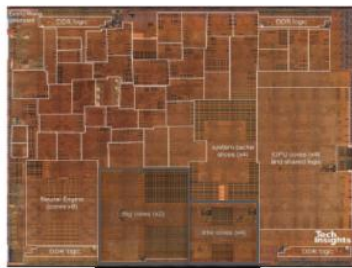
Software **abstracts**
TLP+DLP, e.g.,
CUDA, OpenCL, &
graphics OpenGL
Also **hidden** in
cloud, etc.



Nvidia GK110
BLP+TLP+DLP

Local software stack
abstracts each
accelerator.

**But no good, general
software abstraction
for SoC ALP!**

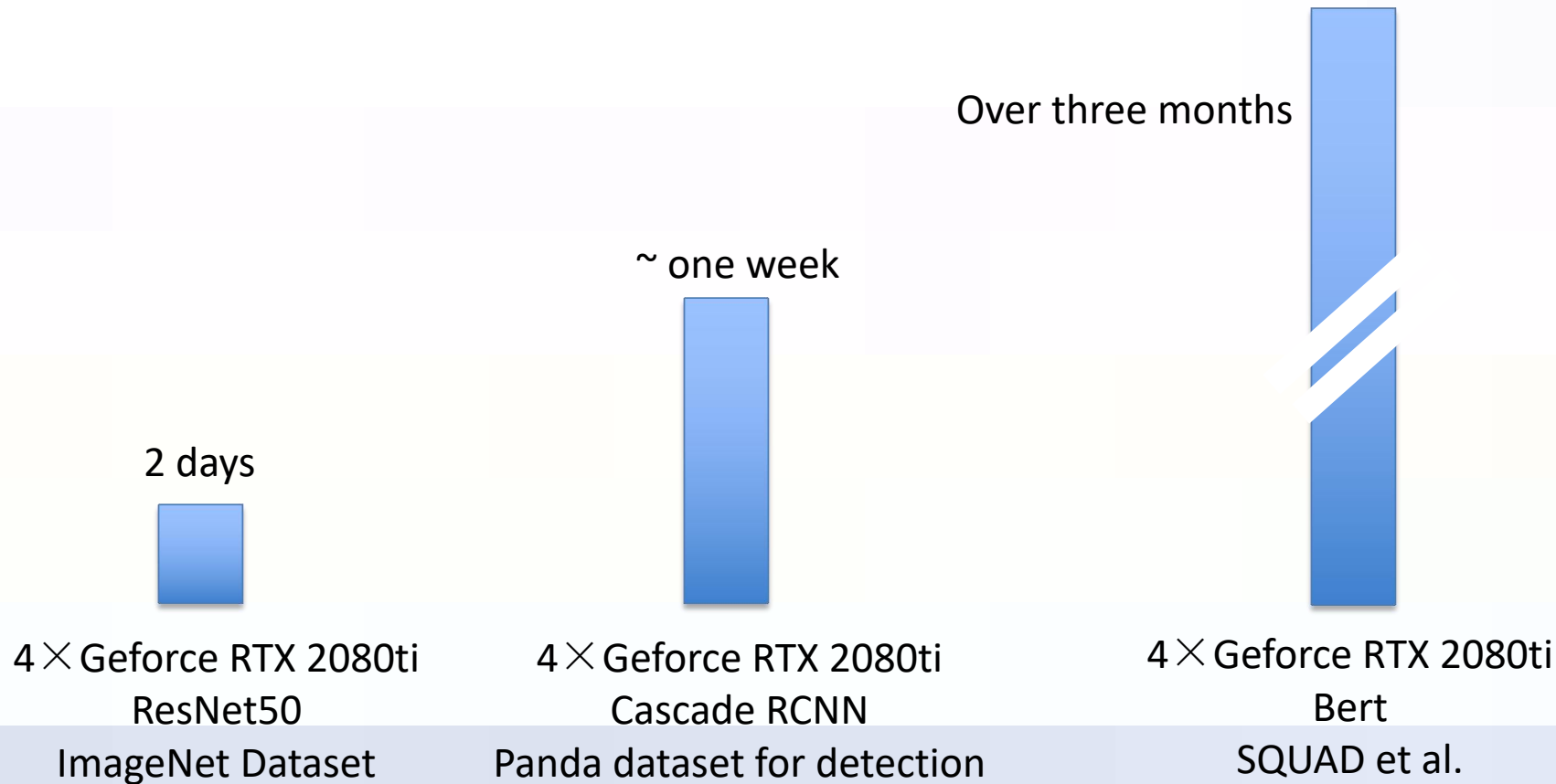


Apple A12
BLP+ILP+TLP+DLP+ALP

/Training Processor

- **Training Flow**
 - **Large training time**
 - **Use better training scheme**
- **Optimizer**
 - **Find better direction**
 - **Adaptive step length**
- **Training Accelerator**
 - **Quantization**
 - **Sparse Training**

/Training

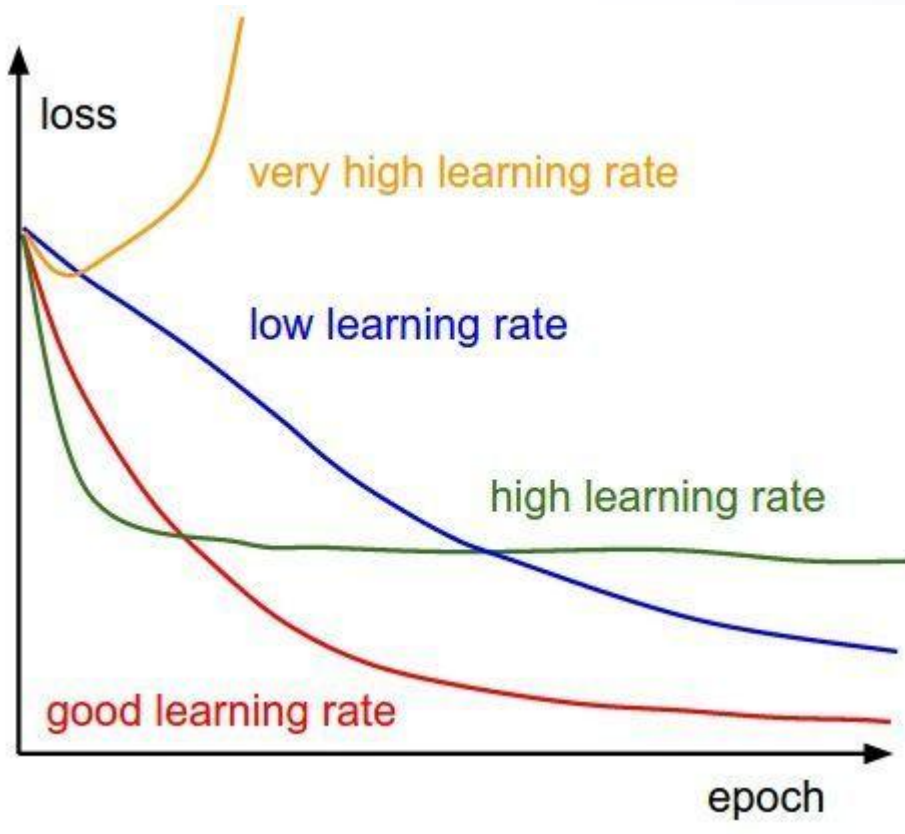


/Training

- Accelerate the training Process is a crucial problem
 - Design a better training scheme
 - Design a better optimizer
 - Design a better training processor

/ Training Scheme

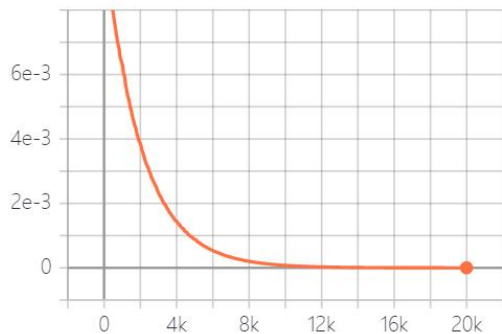
- 学习率直接影响收敛速度和训练时长。
- 在训练初期，使用较大学习率，加快收敛。
- 训练一段时间后，选择较小学习率，使得能够准确逼近最优解。



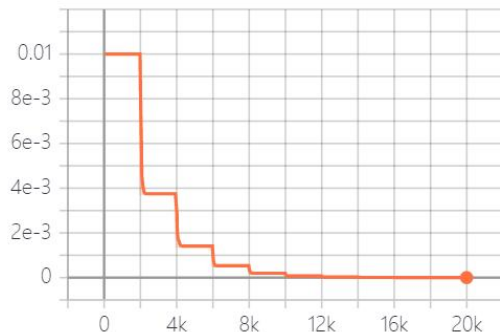
Training Scheme

- 学习率衰减策略（指数、线性、阶梯、余弦等等）

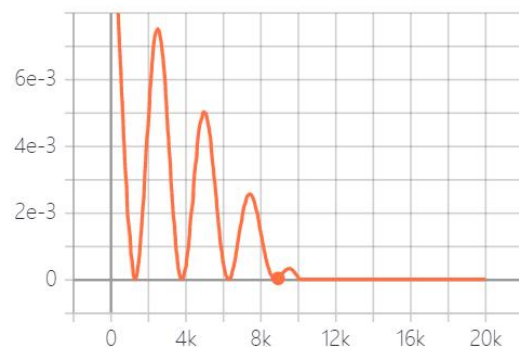
natural_exp_decay_no_stair_1



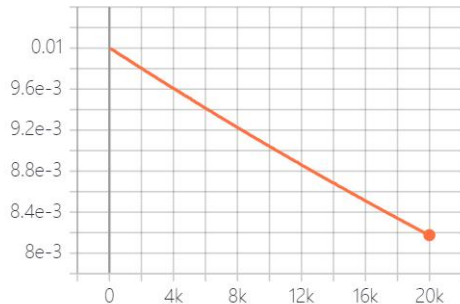
natural_exp_decay_use_stair_1



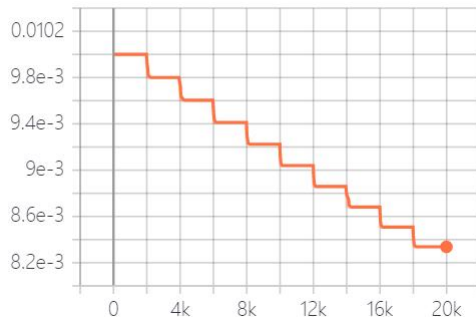
linear_cosine_decay_4_1



exponential_decay_no_stair_1



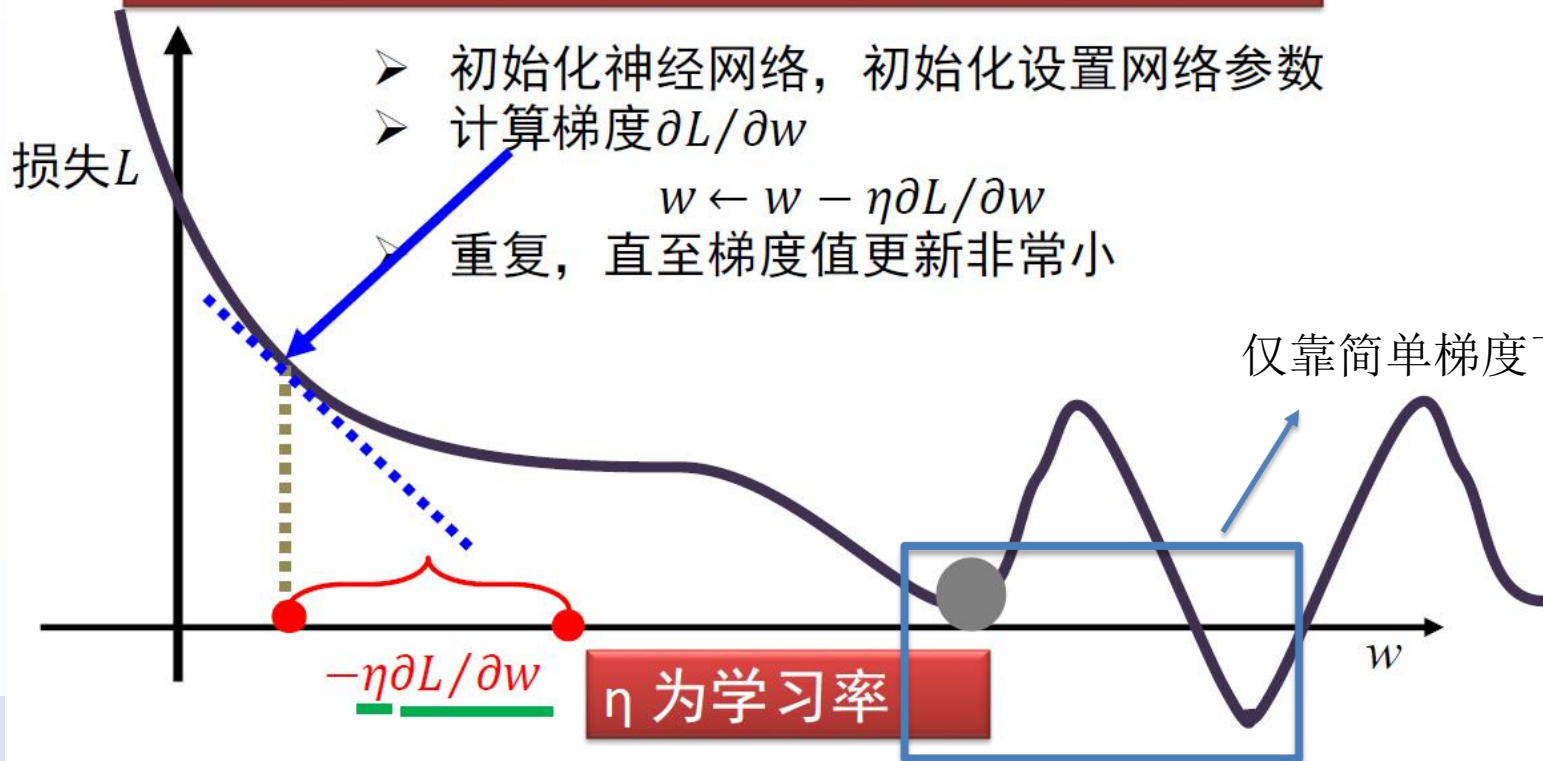
exponential_decay_use_stair_1



- 不同的任务，有不同的适配衰减方法。

/ Training Flow

学习获取使损失函数最小化的网络参数 θ^*



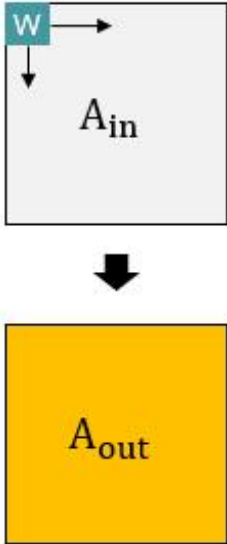
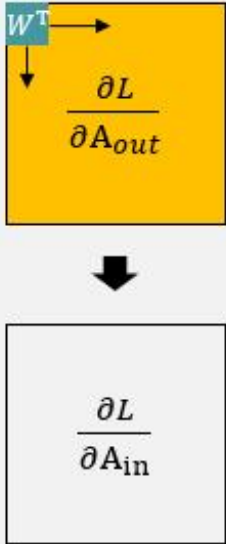
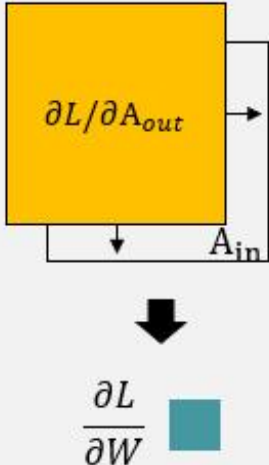
/Optimizer

- 优化器，其设计目的是指引目标函数的各个参数，往**正确的方向更新合适的大小**，使得更新后的参数让损失函数值不断逼近全局最小。
- 一般步骤
 - 计算梯度
 - 计算更新方向及步长
 - 更新参数
- 好的优化器，能够加速收敛，缩短训练时间。

/ Overview

- **Training Flow**
- **Optimizer**
- **Training Accelerator**

/ Analysis

	Forward	Backward	Weight Update
输入	A_{in}, W	$\frac{\partial L}{\partial A_{out}}, W^T$	$\frac{\partial L}{\partial A_{out}}, A_{in}$
输出	A_{out}	$\frac{\partial L}{\partial A_{in}}$	$\frac{\partial L}{\partial W}$
操作	$A_{out} = A_{in} \odot W$ 	$\frac{\partial L}{\partial A_{in}} = \frac{\partial L}{\partial A_{out}} \odot W^T$ 	$\frac{\partial L}{\partial W} = \sum_{i=1}^{Batch} \frac{\partial L}{\partial A_{out}} * A_{in}$ 

/ Training Accelerator

- 训练加速器，前向和反向需要做到一块芯片上。

	Weight	Activation	Gradient
前向	稀疏	稀疏	---
反向	稀疏	稀疏	不稀疏

- 如果利用上节课讲的稀疏计算加速器，会带来什么问题？
 - 前向可加速，反向不能加速，加速效果折半
 - 稀疏加速器有复杂的控制硬件开销，这一切可能就不再值得。
 - 但仍可以考虑权重的稀疏性

课程定位与后续课程（以英伟达的算力发展为例）

Gains from

- Number Representation
 - FP32, FP16, INT8
 - (TF32, BF16)
 - ~16x
- Complex Instructions
 - ~12.5x
- Process
 - 2.5x
- Sparsity
 - ~2x
- Model efficiency has also improved – overall gain > 1000x

数字系统设计

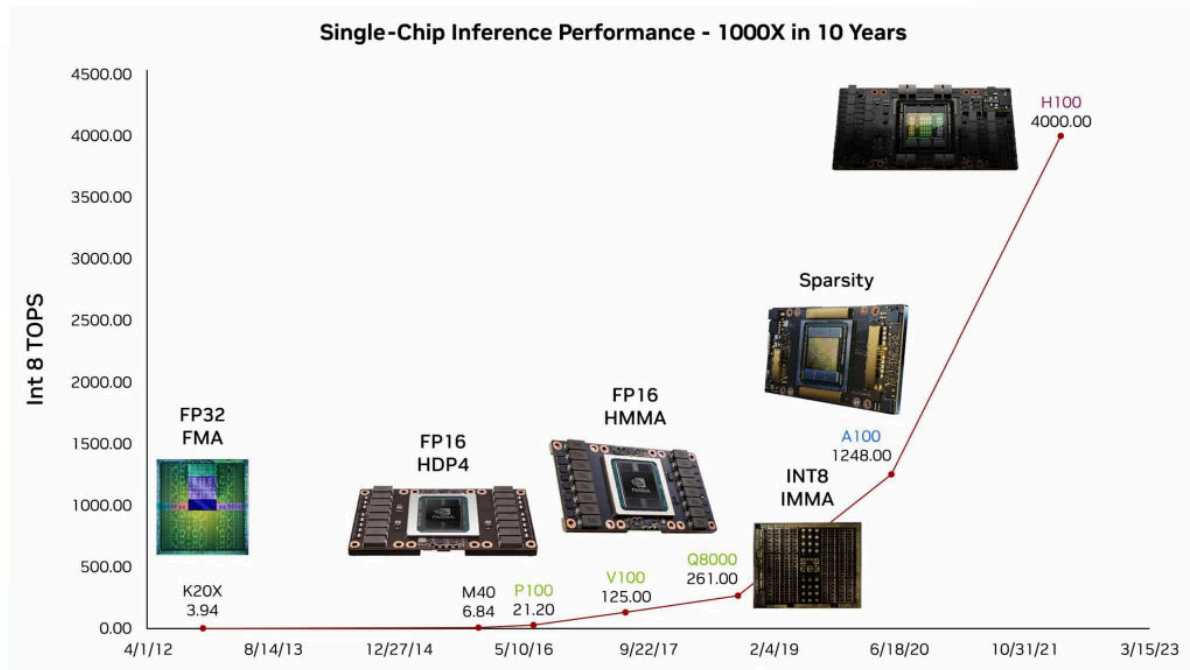
现代体系结构

数字系统设计

半导体器件

数字集成电路

数字系统设计



[Bill Dally, Trends in Deep Learning Hardware]